HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design and Development of a Full-Stack E-Commerce System with Customer Portal and Management Dashboard

Phạm Trường Sang
Sang.PT225996@sis.hust.edu.vn

Program: Global ICT

Supervisor: Dr. Trần Hải Anh

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design and Development of a Full-Stack E-Commerce System with Customer Portal and Management Dashboard

Phạm Trường Sang
Sang.PT225996@sis.hust.edu.vn

Program: Global ICT

Supervisor: Dr. Trần Hải Anh

Signature

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Tran Hai Anh, for your dedicated guidance and for helping me resolve complex technical challenges to complete this thesis. I also sincerely thank the lecturers at Hanoi University of Science and Technology for equipping me with a solid academic foundation over the years.

To my family, my partner, and my friends, thank you for your constant understanding, encouragement, and patience during the stressful late nights I spent working on this project.

Finally, despite my best efforts during the completion of this thesis, certain limitations and shortcomings are inevitable. I sincerely welcome any valuable feedback and recommendations from the examination committee for further improvement.

ABSTRACT

E-commerce systems play an important role in modern retail by allowing customers to access product information, compare alternatives, and purchase products through online platforms. In the context of technology retail, the development of an e-commerce system requires careful handling of product data, since different categories may have different technical specifications and management requirements. At the same time, customer activities such as browsing, ordering, payment, and review submission need to be connected with internal operations such as product management, order processing, inventory control, and user administration. These requirements create the need for an integrated web application that can support both customer interaction and store management in a consistent manner.

This graduation thesis presents the design and development of a full-stack e-commerce system with a customer portal and a management dashboard. The system is built using a client-server approach, where the frontend, backend, database, and external services are organized to support the main workflow of an online retail application. Instead of focusing only on interface construction, the thesis also addresses practical implementation issues that arise during system development, such as managing flexible product specifications, maintaining payment and stock consistency, and improving product data import.

The completed system provides a working application that connects customer purchasing activities with management operations. Through this implementation, the thesis shows how full-stack development, database design, access control, external service integration, and business workflow handling can be combined to build a maintainable e-commerce application. The outcome of the thesis is a functional software product that provides a practical foundation for further improvement, deployment, and real-world operation.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 1

1.1 Motivation 1

1.2 Objectives and scope of the graduation thesis 1

1.3 Tentative solution 2

1.4 Thesis organization 3

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS 4

2.1 Status survey 4

2.2 Functional Overview 5

2.2.1 General use case diagram 6

2.2.2 Detailed use case diagram 8

2.2.3 Business process 12

2.3 Functional description 14

2.3.1 Description of use case Sign Up 14

2.3.2 Description of use case Place Order 15

2.3.3 Description of use case Pay Order 16

2.3.4 Description of use case Cancel Order 17

2.3.5 Description of use case Add Product 18

2.3.6 Description of use case Ban User Account 19

2.4 Non-functional requirements 19

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES 21

3.1 Frontend Technology 21

3.2 Backend Technologies 21

3.2.1 Spring Security and JSON Web Token 21

3.2.2 Caching 22
3.2.3 Server-Sent Events via SseEmitter 22
3.3 Database Technology 23
3.4 External Services and Integrations 23
3.4.1 VNPay 23
3.4.2 Ngrok 23
3.4.3 Cloudinary 23
3.4.4 Email service 24
3.4.5 EasyExcel 24
3.5 Deployment Technology 24
3.5.1 Docker and Docker Compose 24
3.5.2 Nginx 25
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION 26
4.1 Architecture design 26
4.1.1 Software architecture selection 26
4.1.2 Overall design 26
4.1.3 Detailed package design 28
4.2 Detailed design 30
4.2.1 User interface design 30
4.2.2 Layer design 33
4.2.3 Database design 38
4.3 Application Implementation 46
4.3.1 Libraries and Tools 46
4.3.2 Achievement 48
4.3.3 Illustration of main functions 49

4.4 Testing 51
4.4.1 Account Registration and Verification Testing 52
4.4.2 Order and Payment Testing 53
4.4.3 Product Data and Import Testing 54
4.4.4 Access Control and User Management Testing 55
4.5 Deployment 56
CHAPTER 5. SOLUTIONS AND CONTRIBUTIONS 58
5.1 Dynamic Product Attribute Design using PostgreSQL JSONB 58
5.1.1 Problem 58
5.1.2 Proposed Solution 58
5.1.3 Result 60
5.2 VNPay Payment Timeout Recovery and Stock Reservation Cleanup 61
5.2.1 Problem 61
5.2.2 Proposed Solution 62
5.2.3 Result 63
5.3 Memory-Bounded Excel Product Import with Asynchronous Image Upload 63
5.3.1 Problem 63
5.3.2 Proposed Solution 64
5.3.3 Result 65
CHAPTER 6. CONCLUSION AND FUTURE WORK 66
6.1 Conclusion 66
6.2 Future work 66
REFERENCE 69

LIST OF FIGURES

Figure 2.1 Overall Use Case Diagram of the System 6
Figure 2.2 Detailed Use Case Diagram for Browsing Products 8
Figure 2.3 Detailed Use Case Diagram for Managing Cart 9
Figure 2.4 Detailed Use Case Diagram for Customer Order Management 9
Figure 2.5 Detailed Use Case Diagram for Inventory Administration . . 10
Figure 2.6 Detailed Use Case Diagram for Order Fulfillment 11
Figure 2.7 Detailed Use Case Diagram for User Administration 11
Figure 2.8 Activity Diagram of the Order Fulfillment and Payment Process 13

Figure 4.1 UML package diagram of the frontend application 27
Figure 4.2 UML package diagram of the backend application 28
Figure 4.3 UML detailed package diagram of the order placement and VNPay payment 29
Figure 4.4 Structural wireframe of the customer product filtering interface 32
Figure 4.5 Structural wireframe of the Admin and Product Manager workspace 33
Figure 4.6 Class design for order and payment services 34
Figure 4.7 Sequence diagram for the Place Order use case 35
Figure 4.8 Sequence diagram for the Pay Order use case 37
Figure 4.9 Overview E-R diagram of the technology e-commerce system 38
Figure 4.10 Main customer interfaces of the system 49
Figure 4.11 Product detail interface of the system 50
Figure 4.12 Main Product Manager interfaces of the system 50
Figure 4.13 Main Administrator interfaces of the system 51

LIST OF TABLES

Table 2.1 Comparison of similar e-commerce systems 5
Table 2.2 Use Case Specification: Sign Up 14
Table 2.3 Use Case Specification: Place Order 15
Table 2.4 Use Case Specification: Pay Order 16
Table 2.5 Use Case Specification: Cancel Order 17
Table 2.6 Use Case Specification: Add New Product 18
Table 2.7 Use Case Specification: Ban User Account 19

Table 4.1 Target Screen Specifications 30
Table 4.2 Interface Design Conventions 31
Table 4.3 Design of the users table 39
Table 4.4 Design of the user_address table 39
Table 4.5 Design of the refresh_tokens table 40
Table 4.6 Design of the email_verification_tokens table 40
Table 4.7 Design of the notifications table 41
Table 4.8 Design of the notification_read_tracker table 41
Table 4.9 Design of the categories table 41
Table 4.10 Design of the category_attribute_schema table 42
Table 4.11 Design of the category_attribute_group table 42
Table 4.12 Design of the product table 43
Table 4.13 Design of the product_reviews table 43
Table 4.14 Design of the cart table 44
Table 4.15 Design of the cart_detail table 44
Table 4.16 Design of the orders table 45
Table 4.17 Design of the order_detail table 45
Table 4.18 Design of the invoice table 46
Table 4.19 List of development tools and libraries used in the system . . 47
Table 4.20 Implementation statistics of the system 48
Table 4.21 Test cases for account registration and verification 52
Table 4.22 Test cases for order and payment processing 53
Table 4.23 Test cases for order and payment processing (Continued) . . . 54
Table 4.24 Test cases for product filtering and Excel import 54
Table 4.25 Test cases for product filtering and Excel import (Continued) 55
Table 4.26 Test cases for access control and user management 55
Table 4.27 Deployment components of the system 56

LIST OF ABBREVIATIONS

Abbreviation	Full Expression
API	Application Programming Interface
COD	Cash on Delivery
CSRF	Cross-Site Request Forgery attack
DTO	Data Transfer Object
GIN Index	Generalized Inverted Index
JPA	Java Persistence API
JSONB	Binary JSON
JWT	JSON Web Token
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SSE	Server-Sent Events protocol
XSS	Cross-Site Scripting attack

CHAPTER 1. INTRODUCTION

1.1 Motivation

E-commerce has become an important form of modern retail because it changes how customers access product information, compare alternatives, and make purchasing decisions. Instead of depending only on physical stores, customers can search for products, review information, and place orders through online platforms. This creates a practical demand for web systems that can support both online purchasing and store management activities.

In the selected domain of technology retail, product information is usually more complex than ordinary consumer goods. A purchase decision may depend not only on price or brand, but also on technical specifications, product availability, customer reviews, and comparison between similar products. Therefore, the organization and presentation of product information directly affect the customer experience.

From the management perspective, an online store also needs to maintain consistent data about products, categories, stock quantities, orders, payments, reviews, and user accounts. If these activities are handled manually or through separated tools, the process may become time-consuming and more likely to produce inconsistent data.

For these reasons, this thesis addresses the problem of designing and developing a full-stack e-commerce system that combines a customer portal and a management dashboard. The topic is practical because it reflects common requirements of online retail systems and allows the application of software engineering, database design, frontend-backend integration, and external service integration in a complete web application.

1.2 Objectives and scope of the graduation thesis

Existing e-commerce platforms show that online retail is not limited to product display and order creation. A complete system needs to connect customer activities with internal management operations, including product information, inventory, payment status, order processing, user accounts, reviews, and business monitoring. For technology products, this connection becomes more important because product data may include many category-dependent specifications.

In many simple store implementations, these parts are often handled at a basic level or treated as separated functions. Product information may rely on fixed

fields, order processing may not be closely connected with payment status and stock consistency, and management functions may not fully support product data maintenance. These limitations create the need for a more complete application in which customer-facing functions and management functions are designed as parts of the same system.

Based on this context, the objective of this thesis is to design and develop a full-stack e-commerce system with two main parts: a customer portal for online purchasing and a management dashboard for store operation. The system aims to provide an integrated workflow from product discovery to order processing, while also supporting product management, order handling, user administration, review management, and business monitoring.

The scope of the thesis focuses on the core processes required for an e-commerce application, including user access control, product information management, product browsing, cart and order handling, online payment, review management, notification delivery, and administrative monitoring. In addition, the thesis pays attention to several practical implementation issues, such as category-dependent product specifications, interrupted payment flows, stock consistency, and bulk product import.

1.3 Tentative solution

To achieve the stated objective, the system is developed as a full-stack web application based on a client-server architecture. React is used for the frontend to build reusable user interface components for both the customer portal and the management dashboard. Spring Boot is used for the backend to provide RESTful APIs, organize business logic, and integrate security and data access modules.

PostgreSQL is selected as the main database for storing relational business data, while its `jsonb` support is used to handle flexible product attributes for different product categories. The system also applies Spring Security with JWT-based authentication and role-based access control to separate permissions between different user roles.

Several external services are integrated to support practical e-commerce operations. VNPay is used for online payment, Cloudinary is used for product image storage, email service is used for account verification, and Server-Sent Events are used for real-time notifications. Docker Compose and Nginx are used to support deployment and environment consistency.

Overall, the proposed solution aims to build a complete and maintainable e-

commerce application within the scope of a graduation thesis. The expected result is a working full-stack system that connects customer purchasing activities with store management operations and applies selected technical solutions for dynamic product information, payment handling, and product data import.

1.4 Thesis organization

The remainder of this graduation thesis is organized as follows.

Chapter 2 presents the requirement survey and analysis of the system. This chapter identifies the current status of the problem, defines the main functional and non-functional requirements, and describes the system behavior through use case diagrams, business processes, and selected use case descriptions.

Chapter 3 introduces the theoretical background and technologies used to develop the system. This chapter explains the main technology groups applied in the project, including frontend development, backend development, database management, external service integration, and deployment support. The purpose of this chapter is to provide the technical foundation for the design and implementation presented in the following chapters.

Chapter 4 presents the design, implementation, and evaluation of the system. This chapter describes the overall architecture, detailed system design, database design, and user interface design. It also presents the implementation results, illustrates the main functions of the application, and evaluates important workflows through testing and deployment.

Chapter 5 presents the main solutions and contributions of the thesis. This chapter focuses on selected technical problems addressed during the development process, including flexible product attribute management, payment timeout recovery with stock reservation cleanup, and efficient product import. Each contribution is presented through its problem, proposed solution, and achieved result.

Chapter 6 concludes the thesis and presents future work. This chapter summarizes the main results of the project, discusses the remaining limitations, and proposes possible directions for improving the system in future development.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

2.1 Status survey

To identify the main requirements of the system, a survey was conducted on several existing e-commerce platforms and technology retail websites, including Shopee, Tiki, CellphoneS, and The Gioi Di Dong. Shopee and Tiki were selected as representatives of general e-commerce platforms, while CellphoneS and The Gioi Di Dong were selected because they focus more on technology product retail. The survey was based on publicly observable customer-facing pages and publicly available seller or support documentation [1], [2], [3], [4], [5], [6].

The surveyed systems show that common e-commerce functions, such as product search, filtering, online payment, and product feedback, are widely supported. However, the level of support differs between general marketplaces and technology retail websites. General e-commerce platforms usually provide broad filtering criteria, while technology retail websites tend to provide more detailed product specifications and product comparison functions. Some management-related functions, such as inventory management, dashboard-based operation, Excel-based product import, and dynamic product attribute management, are not always visible from public customer interfaces. Therefore, the comparison uses documentation-based or not publicly observable labels when these functions cannot be directly verified from public pages.

Table 2.1 summarizes the comparison between similar systems and the system developed in this thesis. In the table, “Yes” means the function is publicly observable, “Limited” means the function is supported but not detailed or not strongly focused, “Not found” means the function was not found during the public survey, “Doc.” means the function is mentioned in public seller documentation, and “N/O” means the function is not publicly observable.

As shown in Table 2.1, existing systems provide strong support for common customer-facing functions. However, product filtering and product comparison are more suitable for technology retail websites than for general e-commerce platforms, because customers who purchase technology products often need to compare technical specifications before making a decision.

From the management perspective, several functions are not fully visible from public interfaces. Marketplace platforms provide seller documentation for some management activities, while technology retail websites mainly expose customer-

facing functions. Therefore, the survey is used as a reference for identifying system requirements rather than making absolute conclusions about the internal implementation of the surveyed platforms.

Criteria	**Shopee**	**Tiki**	**CellphoneS**	**TGDD**	**This thesis**
Product search and filtering	Limited	Limited	Yes	Yes	Yes
Product specification comparison	Not found	Not found	Yes	Yes	Yes
Online payment	Yes	Yes	Yes	Yes	Yes
Product reviews or feedback	Yes	Yes	Yes	Yes	Yes
Inventory management	Doc.[2]	Doc.[4]	N/O	N/O	Yes
Management dashboard	Doc.[2]	Doc.[4]	N/O	N/O	Yes
Excel-based product import	Doc.[2]	Doc.[4]	N/O	N/O	Yes
Dynamic product attribute management	N/O	N/O	N/O	N/O	Yes

Table 2.1: Comparison of similar e-commerce systems

Based on the survey, the system developed in this thesis should support both customer purchasing activities and internal management operations. The important functions include product browsing, filtering, comparison, ordering, payment, review submission, product management, order management, user management, revenue monitoring, dynamic product attributes, and bulk product import. These requirements are used as the basis for the functional overview and use case analysis in the following sections.

2.2 Functional Overview

The software system is designed to provide a specialized e-commerce platform that serves the entire lifecycle of distributing and purchasing technology devices. From the consumer perspective, the application fulfills core needs including product retrieval via dynamic technical attribute filters, online shopping cart management, and transaction completion through an electronic payment gateway. From the business operation perspective, the system integrates specialized administrative modules to automate the inventory data entry process, moderate delivery statuses,

and analyze business performance. All of these functional flows are governed by a strict authentication and authorization mechanism, ensuring data security and optimizing the interaction experience for each specific user group.

2.2.1 General use case diagram

Figure 2.1 presents the overall use case diagram of the e-commerce system. The diagram illustrates the interactions between the system and four main actors, including Guest, Customer, Product Manager (PM), and Administrator (Admin). Each actor is responsible for a different set of functionalities within the system.

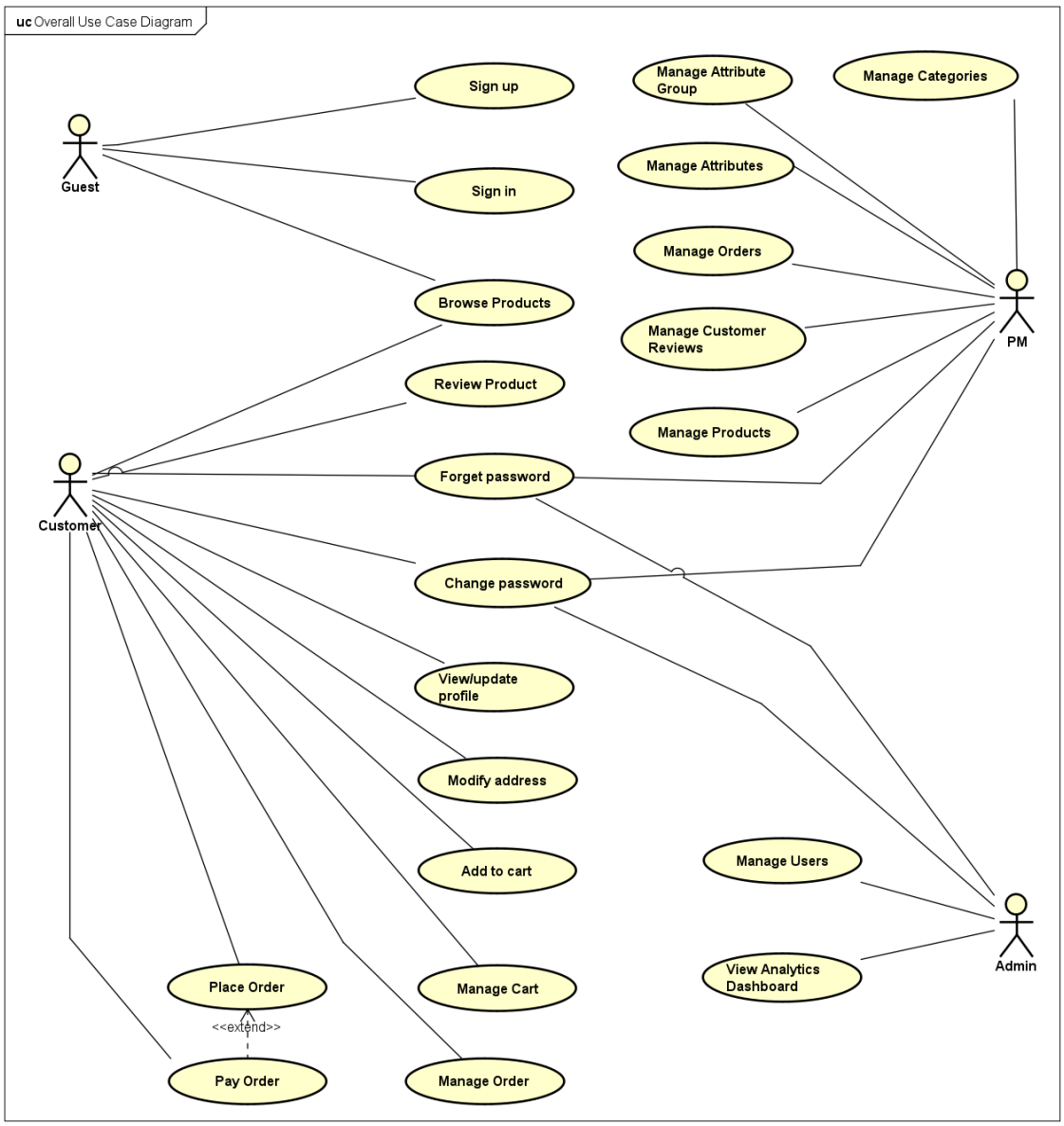


Figure 2.1: Overall Use Case Diagram of the System

The Guest actor represents visitors who access the website without authentication. Guests can browse products and use basic features before creating an account or logging into the system.

The most important use cases of the Guest actor are Sign Up, Sign In, and Filter Products. The Sign Up use case allows new users to create a new account and gain access to customer features within the system. The Sign In use case enables registered users to authenticate their accounts and access functionalities according to their assigned roles, including Customer, Product Manager (PM), or Administrator (Admin). In addition, the Filter Products use case helps users search for products more efficiently by applying criteria such as category, brand, or price range.

The Customer actor represents registered users who have successfully logged into the system. Customers can purchase products and interact with the platform through several business functions.

Among all customer functionalities, Place Order, Pay Order, and Product Review are the most important use cases. Place Order allows customers to submit purchase requests for selected products. Pay Order handles the transaction process through supported payment methods. After receiving products, customers can provide ratings and feedback through the Product Review use case.

The Product Manager (PM) is responsible for maintaining product information and processing customer orders.

The important use cases of the Product Manager are Add Product, Manage Orders, and Manage Attributes. The Add Product use case allows the Product Manager to create and maintain product information in the system, including basic product details, stock quantity, images, and technical specifications. The Manage Orders use case allows the Product Manager to monitor order status and perform order-processing activities such as confirmation, preparation, and delivery handling. The Manage Attributes use case supports the management of product specification structures for different product categories. Attributes represent individual technical characteristics, such as RAM, processor, screen size, or battery capacity. These use cases help the Product Manager maintain accurate product data and support consistent order processing.

The Administrator actor is responsible for system administration and business monitoring.

The most important administrator functionalities are Manage Users and View Revenue. Manage Users allows administrators to monitor user accounts and control account status when necessary. View Revenue provides access to revenue statistics and sales reports, helping administrators evaluate the overall performance of the e-commerce system.

2.2.2 Detailed use case diagram

a, Detailed use case: Browse Product (Customer)

This functional group defines how the Customer interacts with the retail interface to discover technology components. As illustrated in Figure 2.2, the core operation is browsing the product repository. To enhance the retrieval accuracy, this main use case is dynamically extended by advanced searching capabilities and filtering mechanisms based on specific technical attributes. Furthermore, users are provided with extended options to compare multiple hardware specifications side-by-side or view detailed product descriptions before making a purchasing decision.

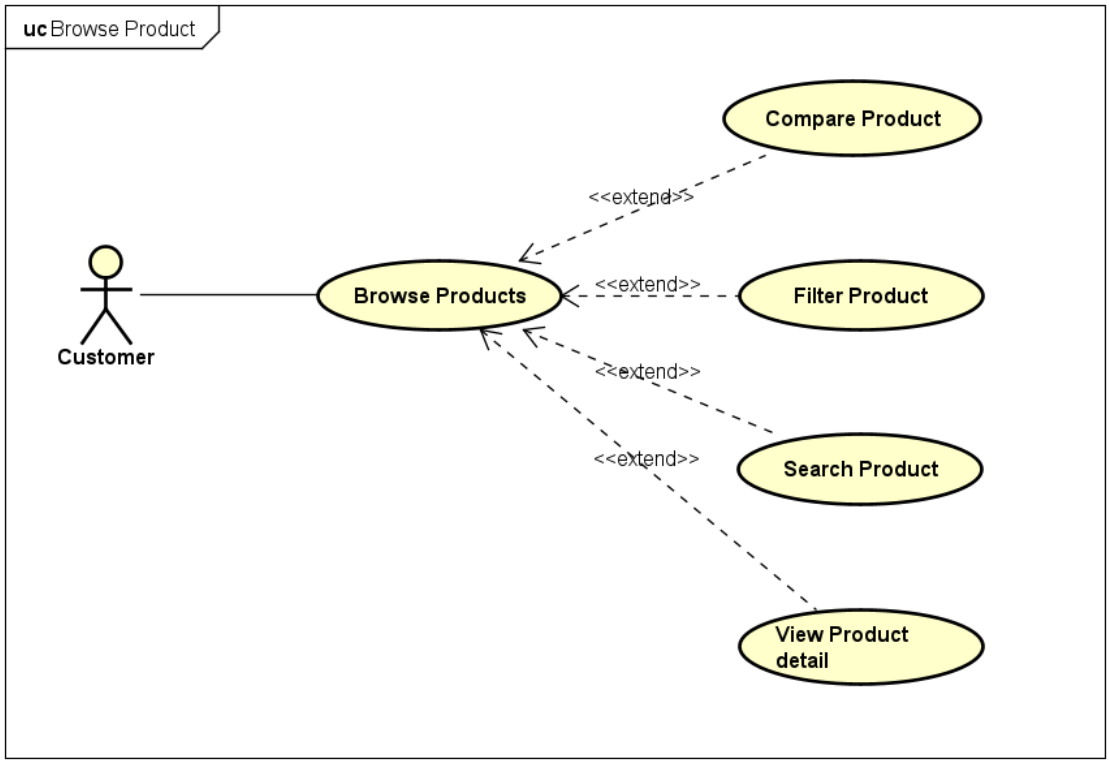


Figure 2.2: Detailed Use Case Diagram for Browsing Products

b, Detailed use case: Manage Cart (Customer)

The shopping cart management workflow handles the pre-checkout data aggregation for Customers. Figure 2.3 demonstrates the central activity of accessing and managing the cart interface. From this primary state, the system allows users to extend their actions by updating the quantity of specific items, removing unwanted technology products from the current session, or viewing the detailed breakdown of the cart contents prior to initiating the financial transaction.

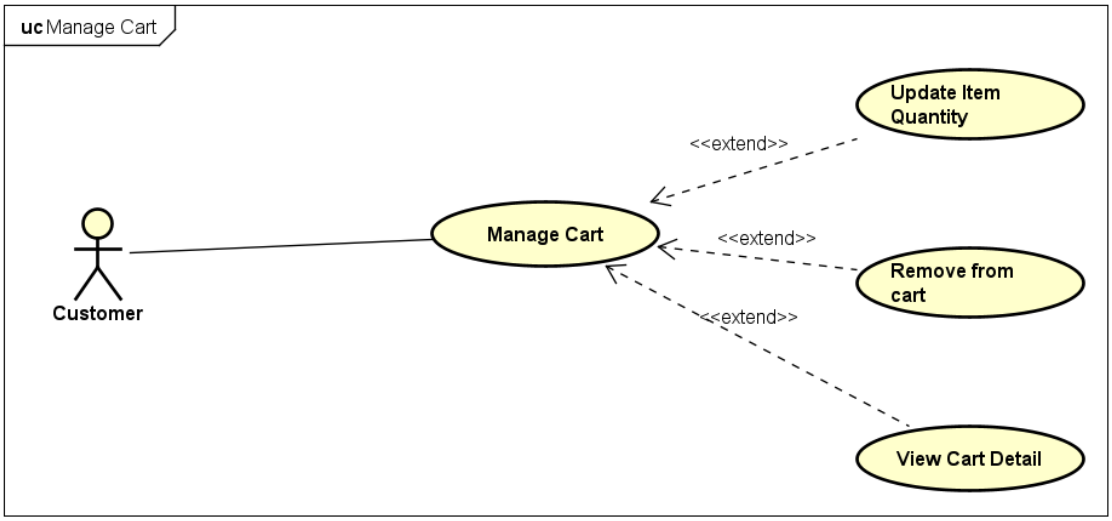


Figure 2.3: Detailed Use Case Diagram for Managing Cart

c, Detailed use case: Manage Order (Customer)

Following a successful checkout process, Customers require tracking tools to monitor their purchases. As depicted in Figure 2.4, the primary operation involves managing personal orders. This functionality is structurally extended by secondary use cases that allow the Customer to view their comprehensive order history, inspect specific transaction details, continue with payments, or filter past orders by status. Additionally, the system provides an extended workflow for canceling an order, provided the delivery state machine has not yet been finalized.

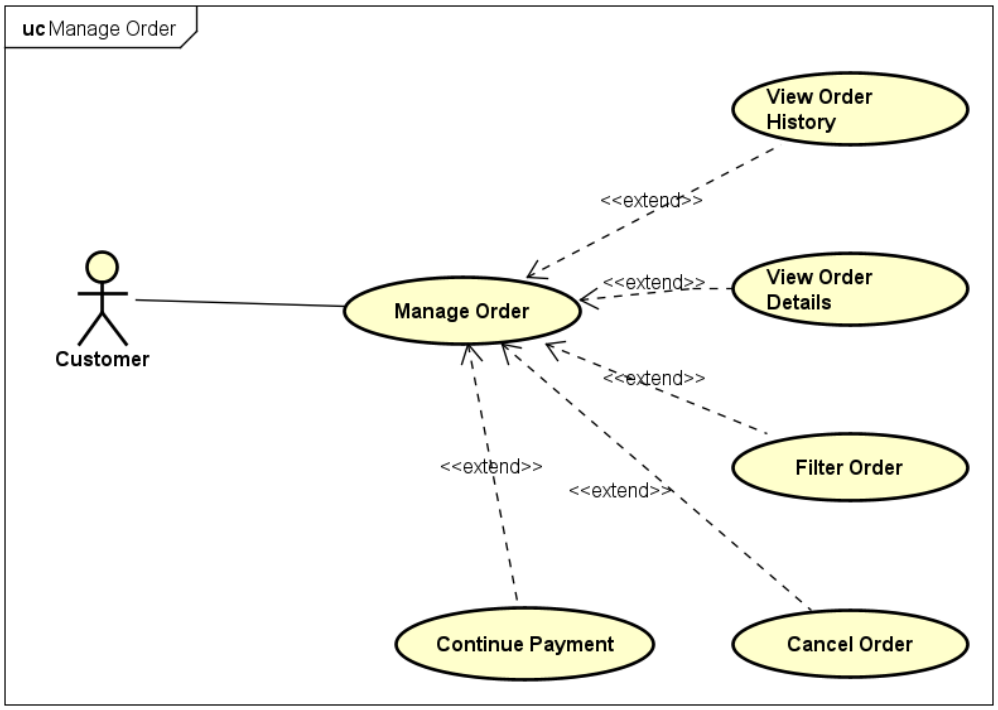


Figure 2.4: Detailed Use Case Diagram for Customer Order Management

d, Detailed use case: Manage Product (Product Manager)

The Product Manager is responsible for maintaining product data in the system. Figure 2.5 presents the detailed product management functions. This actor can add new products, update existing product information, and remove obsolete records when necessary. To support large-scale data entry, the system provides an extended function for importing multiple products from Excel spreadsheets. The product list view and filtering functions also help the Product Manager monitor product information and stock availability more effectively.

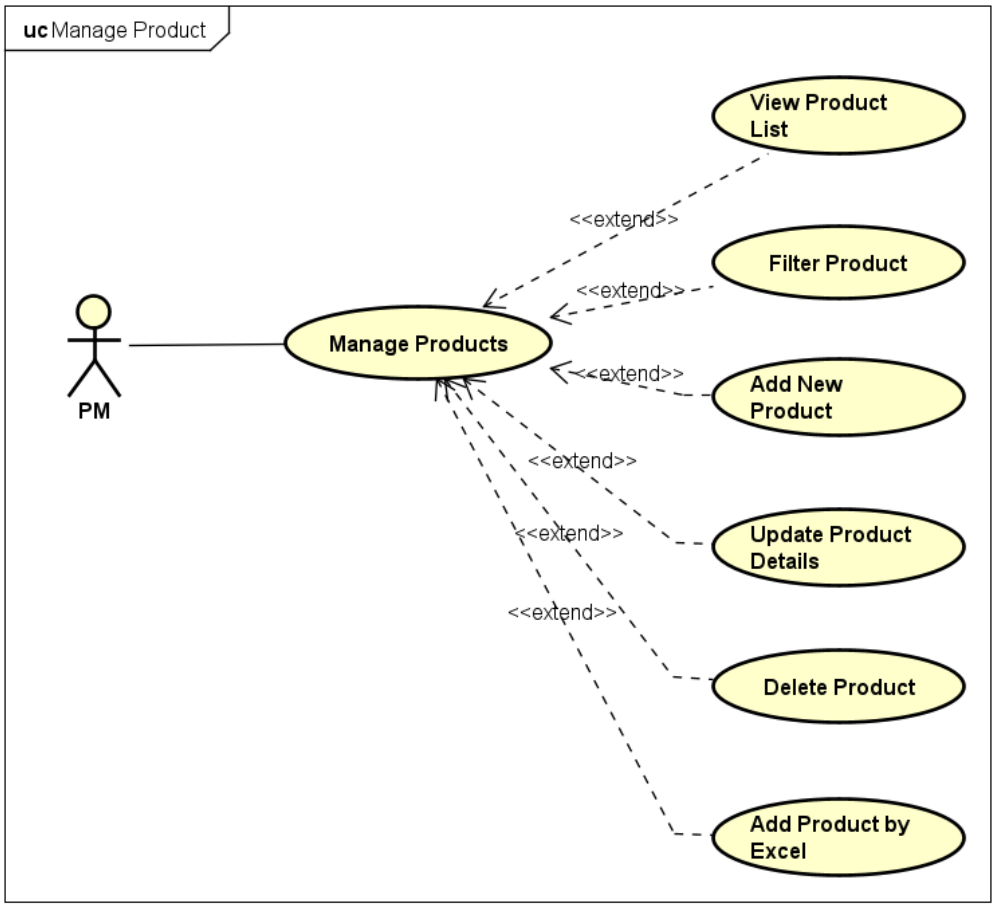


Figure 2.5: Detailed Use Case Diagram for Inventory Administration

e, Detailed use case: Manage Orders (Product Manager)

On the administrative side, the Product Manager must oversee the global fulfillment lifecycle of all customer transactions. As captured in Figure 2.6, this operational domain revolves around managing the centralized order queue. The Product Manager relies on extended use cases to view the consolidated list of incoming purchases, filter transactions based on their current operational states, and manually process orders to update the delivery progress within the system.

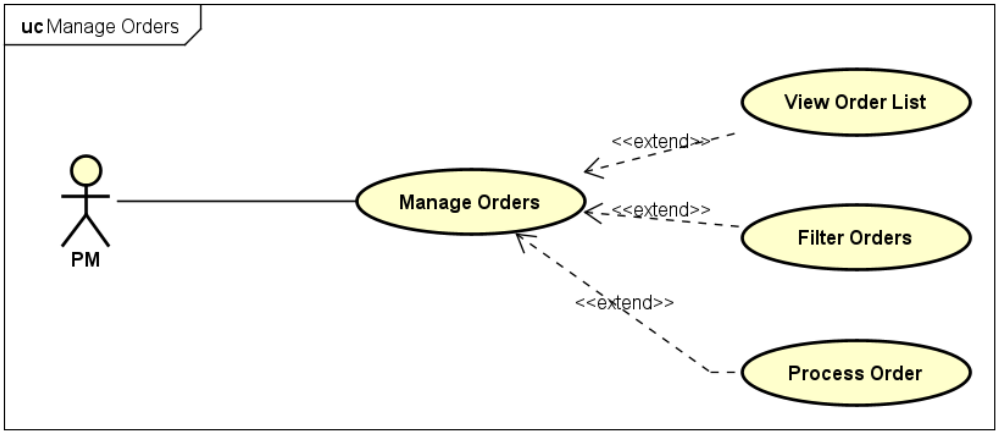


Figure 2.6: Detailed Use Case Diagram for Order Fulfillment

f, Detailed use case: Manage Users (Admin)

The Admin actor is responsible for platform security and personnel access oversight. Figure 2.7 breaks down the user management workflow into specific administrative actions. Operating from the central user management interface, the Admin can browse the registered account list and dynamically update user roles to delegate system privileges. Furthermore, this workflow extends to enforcing strict account sanctions by banning violating entities, manually provisioning new operational accounts, or permanently deleting user records from the database to ensure system integrity.

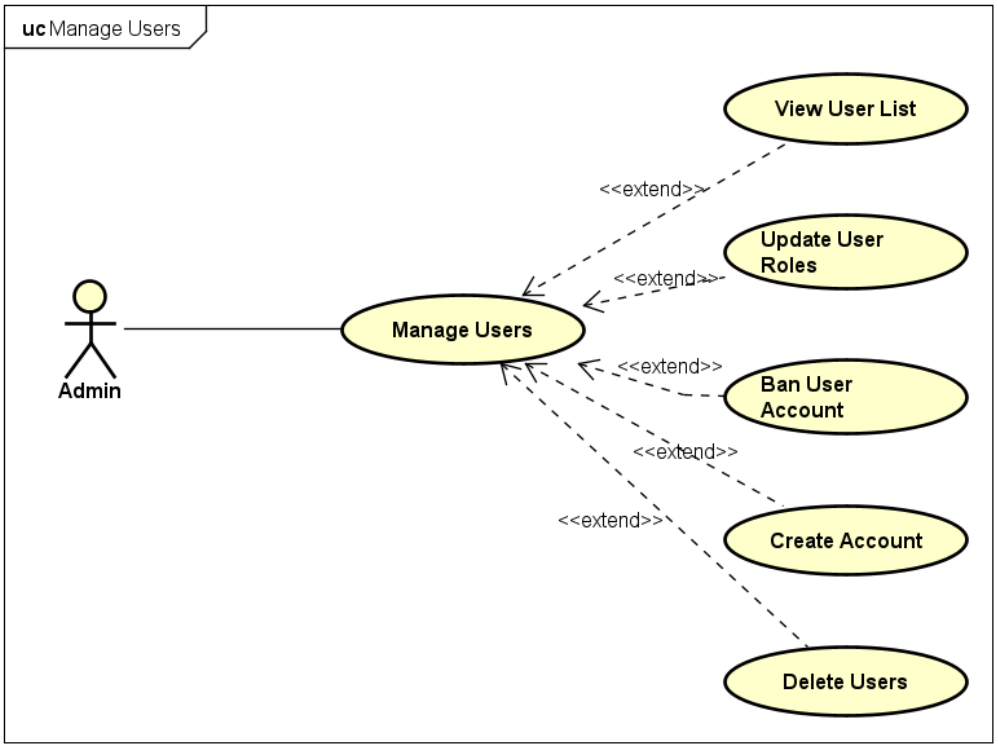


Figure 2.7: Detailed Use Case Diagram for User Administration

2.2.3 Business process

As shown in Figure 2.8, the order fulfillment process begins when the Customer reviews the cart, provides delivery information, selects a payment method, and places the order. The System then checks product availability. If the requested products are out of stock, the process is terminated. Otherwise, the System creates the order and processes the selected payment method. For VNPay, the Customer is required to complete the online payment step, while Cash on Delivery (COD) does not require immediate payment. After the payment information is recorded, the System sends a notification to the Product Manager.

The Product Manager then reviews the order and decides whether to approve it. If the order is rejected, the System notifies the Customer about the cancellation. If the order is approved, the Product Manager monitors the delivery process. The workflow ends when the delivery is either marked as failed or the order is successfully completed.

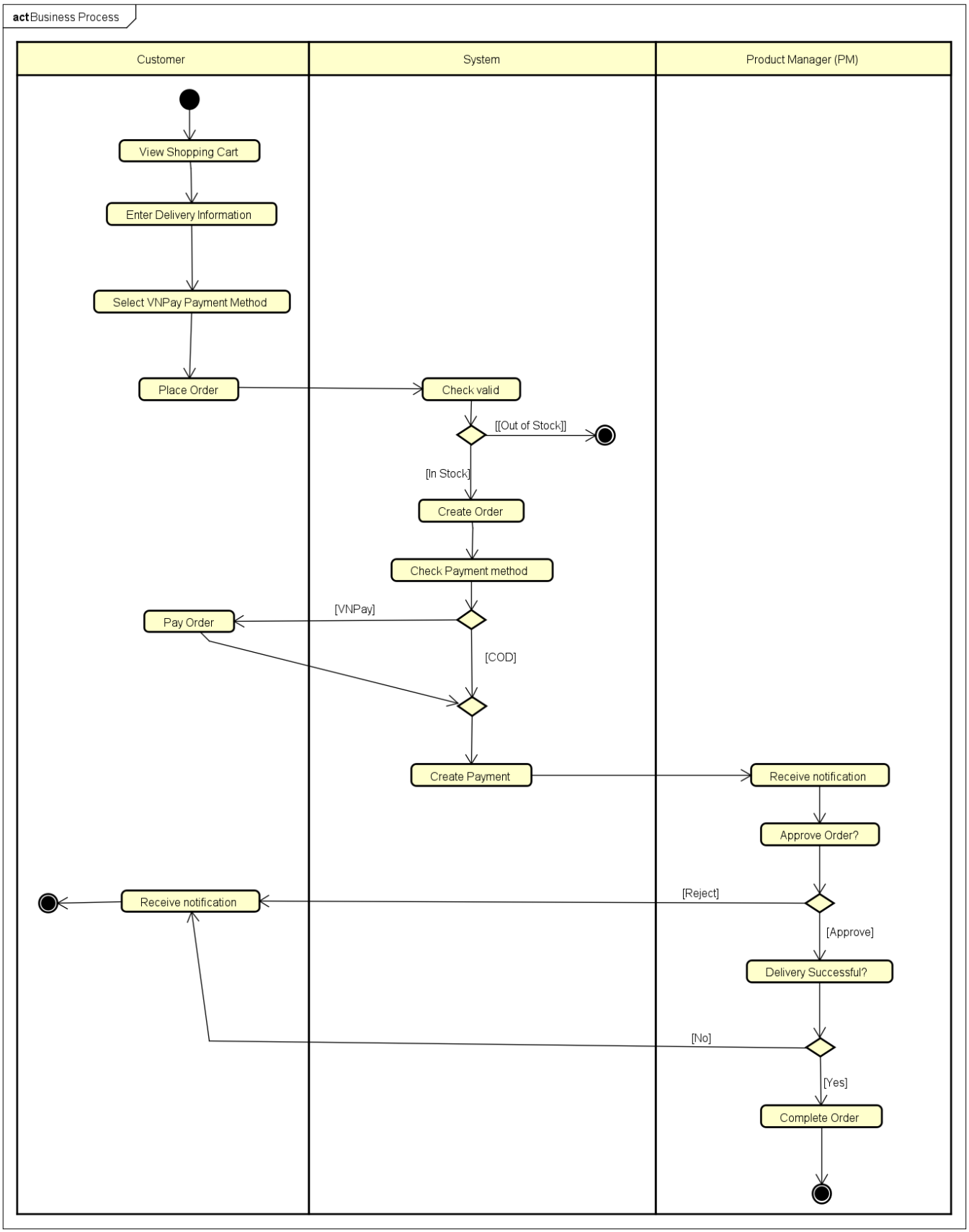


Figure 2.8: Activity Diagram of the Order Fulfillment and Payment Process

2.3 Functional description

2.3.1 Description of use case Sign Up

Use Case Name	Sign Up
Actor	Guest
Preconditions	The user is not authenticated in the system.
Description	This use case allows a Guest to create a new customer account with email OTP verification.
Main Flow	1. The Guest enters registration information, including username, email, password, and confirm password. 2. The System validates the submitted information, such as duplicate email or username, password format, and password confirmation. 3. The System creates an inactive account and sends an OTP to the registered email address. The OTP is valid for 10 minutes. 4. The Guest enters the OTP on the verification form. 5. The System verifies the OTP and checks whether it is still valid. 6. The System activates the account. 7. The System displays a success message and redirects the user to the Sign In page.
Alternative Flow	2a. If the email or username already exists, or if the password and confirm password do not match, the System displays an error message. 2b. If the password does not satisfy the password policy, the System displays an error message. The password must be at least 8 characters and include uppercase, lowercase, digit, and special characters. 3a. If the Guest requests a new OTP before the 60-second resend cooldown period ends, the System rejects the request and displays an info message. 3b. After 24 hours, if the account remains inactive, the System scans and deletes the unverified account. 5a. If the OTP is incorrect, the System displays an error message and records the failed attempt. 5b. If the number of failed attempts exceeds 5 or the OTP expired, the System invalidates the current OTP and requires the Guest to request a new OTP.
Postconditions	A new customer account is created and activated successfully. The user can sign in to the system.

Table 2.2: Use Case Specification: Sign Up

2.3.2 Description of use case Place Order

Use Case Name	Place Order
Actor	Customer
Preconditions	The Customer is authenticated and has at least one item in the cart.
Description	This use case allows a Customer to place an order.
Main Flow	1. The Customer submits delivery information and selects a payment method. 2. The System validates the submitted order form. 3. The System checks that the cart is not empty. 4. The System checks that all products in the cart are active. 5. The System verifies that the requested quantities are available in stock. 6. The System calculates the delivery fee. 7. The System creates a new order record. 8. The System decreases the product quantities in inventory. 9. The System checks the payment method.If the customer opts for Cash on Delivery (COD), the cart is cleared. 10. The System creates a payment record. 11. The System sends a real-time notification to the Product Manager.
Alternative Flow	2a. If the order form is invalid, the System stops the process and asks the Customer to correct the information. 3a. If the cart is empty, the System displays a message and terminates the process. 4a. If any product is inactive, the System displays an invalid product message and terminates the process. 5a. If stock is insufficient, the System displays an insufficient quantity message and terminates the process. 9a. If the Customer selects an online payment method such as VNPay, the System creates a pending payment record and starts the Pay Order process to handle the electronic payment. 10a. If the VNPay payment period expires while the payment status remains pending, the System detects the expired payment, marks it as failed, restores the reserved stock, and cancels the pending order if necessary.
Postconditions	A new order is created, inventory is updated, the Product Manager receives a notification, and the Customer either completes the Cash order or continues to the electronic payment flow.

Table 2.3: Use Case Specification: Place Order

2.3.3 Description of use case Pay Order

Use Case Name	Pay Order
Actor	Customer
Preconditions	The Customer has placed an order processed via VNPay, and the payment is still pending.
Description	This use case describes how the Customer completes an online payment through VNPay. The return URL is used to verify and display the payment result to the Customer, while the verified IPN callback is used to update the final payment status in the database.
Main Flow	1. The System creates a VNPay payment request based on the order information. 2. The System redirects the Customer to the VNPay payment page. 3. VNPay displays the secure payment interface. 4. The Customer enters payment information and confirms the payment. 5. VNPay processes the transaction and sends the result through the return URL and IPN callback. 6. The System verifies the return URL data and displays the payment result to the Customer. 7. The System receives IPN data from VNPay and checks the IPN secure hash, transaction reference, payment amount, and current payment status. 8. If the payment is still pending, the System updates the payment status based on the verified IPN result and returns a confirmation response to VNPay.
Alternative Flow	6a. If the return URL data is invalid, the System displays an error message and does not treat the result as trusted. 7a. If the IPN secure hash is invalid, the System rejects the IPN request and does not modify the payment record. 7b. If the payment record is not found, the System returns an order-not-found response to VNPay. 7c. If the payment amount does not match the stored payment amount, the System returns an invalid-amount response to VNPay. 8a. If the payment record has already been processed, the System ignores the repeated update request. 8b. If the payment fails or is canceled, the System marks the payment as failed, restores the reserved stock, and cancels the pending order.
Postconditions	The Customer receives the payment result, and the final payment status is updated only once based on the verified IPN callback.

Table 2.4: Use Case Specification: Pay Order

2.3.4 Description of use case Cancel Order

Use Case Name	Cancel Order
Actor	Customer
Preconditions	The Customer is authenticated and has an order that is still allowed to be canceled.
Description	This use case allows a Customer to cancel an existing order. The System checks the order status and processes a refund through VNPay if the order has already been paid.
Main Flow	1. The Customer selects an order and chooses the cancel option. 2. The System displays a confirmation dialog. 3. The Customer confirms the cancellation. 4. The System checks the current status of the order. 5. The System determines that the order is still pending and has not been paid. 6. The System updates the order status to “Canceled”. 7. The System saves the change to the database and displays a successful cancellation message.
Alternative Flow	3a. If the Customer does not confirm the cancellation, the System closes the dialog and makes no changes. 5a. If the order has already been paid, the System requests a refund through VNPay before updating the order status to “Canceled”. 5b. If the order is already shipping or completed, the System rejects the cancellation request and displays an error message.
Postconditions	The order is canceled in the database, the Customer is notified, and a refund is processed through VNPay if required.

Table 2.5: Use Case Specification: Cancel Order

2.3.5 Description of use case Add Product

Use Case Name	Add Product
Actor	Product Manager (PM)
Preconditions	The Product Manager is authenticated and has permission to access the product management dashboard.
Description	This use case allows the Product Manager to add a new product to the catalog. The Product Manager selects a category, enters basic product information, fills in category-based attributes, and uploads product images.
Main Flow	1. The Product Manager selects the option to add a new product. 2. The System displays the product creation form. 3. The Product Manager selects a product category. 4. The System loads the corresponding attributes of the selected category. 5. The Product Manager enters basic product information, such as name, price, stock quantity, and description. 6. The Product Manager fills in the required attribute values. The values may be in different formats, such as number, list, or string. 7. The Product Manager uploads product images. 8. The System validates the submitted product information and attribute values. 9. The System uploads the product images to Cloudinary for storage. 10. The System saves the new product, attribute values, and image URLs to the database. 11. The System displays a successful creation message to the Product Manager.
Alternative Flow	8a. If the product information or attribute values are invalid, the System displays error messages and asks the Product Manager to correct the input. 9a. If the image upload fails, the System continues normally and notifies the Product Manager about the upload error.
Postconditions	A new product is created in the database with its category, attribute values, and image URLs stored successfully.

Table 2.6: Use Case Specification: Add New Product

2.3.6 Description of use case Ban User Account

Use Case Name	Ban User Account
Actor	Admin
Preconditions	The Admin is authenticated and has permission to manage user accounts.
Description	This use case allows the Admin to ban a user account that violates system rules or platform policies. After being banned, the user is no longer allowed to access protected system functions.
Main Flow	1. The Admin opens the user management page and searches for the target user account. 2. The Admin selects the account and chooses the ban option. 3. The System displays a confirmation dialog and asks the Admin to confirm the action. 4. The Admin confirms the ban action. 5. The System updates the target account status to "Banned" in the database. 6. The System revokes active login sessions by invalidating related JWT tokens and session cookies. 7. The System updates the user management interface and displays a successful notification to the Admin.
Alternative Flow	4a. If the Admin cancels the confirmation dialog, the System stops the process and does not change the user account status.
Postconditions	The selected user account is marked as banned, active sessions are terminated, and the user is blocked from accessing protected system functions.

Table 2.7: Use Case Specification: Ban User Account

2.4 Non-functional requirements

This section presents the non-functional requirements of the e-commerce platform, including security, performance, reliability, maintainability, and usability. For security, the system must protect user-specific and administrative operations through secure authentication and Role-Based Access Control (RBAC). Each actor, including Customer, Product Manager, and Administrator, can only access functions that match their assigned permissions. In addition, all client requests must be validated before backend processing to prevent invalid or harmful data from entering the system.

For performance and reliability, the system must provide a smooth user experience with fast page navigation. The frontend uses a single-page architecture to

reduce unnecessary server requests. The backend must handle concurrent transactions while maintaining data consistency in the relational database. Containerization is also required to ensure that the system can run consistently across development and production environments.

For maintainability, the source code must follow a layered architecture that separates controllers, business logic, and data access. This structure makes the system easier to maintain and extend. For usability, the platform should provide real-time notifications for order updates and system alerts without requiring manual page refreshes. Large media files, such as product images, should be stored on an external cloud service to reduce local server storage and improve delivery performance.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter presents the main theoretical background and technologies used to develop the technology e-commerce website. The discussion focuses only on major technologies that directly support the requirements analyzed in Chapter 2. For each technology, this chapter explains its role in the system, possible alternative approaches, and the reason why it was selected for this project.

3.1 Frontend Technology

The frontend of the system is developed using React [7]. It is responsible for building the user interface through reusable components and providing a smooth interaction flow for different user roles. This is suitable for the e-commerce platform because most user actions require frequent page navigation and dynamic data updates. React helps organize these interfaces in a maintainable and extensible way.

Several alternative frontend technologies could be used, such as Vue.js, or Angular. However, React was selected because it is suitable for building a single-page application and provides good support for component-based interface development.

3.2 Backend Technologies

The backend of the platform is developed using the Spring Boot framework, which is responsible for building RESTful APIs, processing core business logic, handling authentication, and managing database connectivity [8]. Spring Boot was selected because it provides efficient native integration with essential ecosystem modules, such as Spring Security for access control, Spring Data JPA for database operations, and built-in request validation mechanisms. These integrations significantly reduce boilerplate configuration, streamline the overall development process, and ensure a highly consistent and maintainable backend structure.

3.2.1 Spring Security and JSON Web Token

To establish a secure access boundary, the backend integrates Spring Security in conjunction with JSON Web Tokens (JWT) to govern the complete authentication and authorization lifecycle [8], [9]. Spring Security functions as a robust servlet filter chain that intercepts incoming client requests to enforce Role-Based Access Control (RBAC), ensuring that sensitive administrative and product management endpoints are strictly restricted based on granted user privileges. Complementing this, JWT acts as a stateless, cryptographically signed token mechanism used to securely carry authenticated user identity claims across the network layer. This

combination is selected because it eliminates the overhead of server-side session storage. To optimize security, ephemeral access tokens are maintained in-memory to thwart Cross-Site Request Forgery (CSRF), while long-lived refresh tokens are encapsulated within HttpOnly, Secure persistent cookies to effectively eliminate client-side Cross-Site Scripting (XSS) data theft risks.

3.2.2 Caching

To optimize system throughput and eliminate database query latency, the platform leverages the Spring Cache abstraction backed by the Caffeine caching provider [8], [10]. This infrastructure is deployed to store highly requested, semi-static data structures, such as technology product categories and product attributes directly inside the application server. While Spring Cache provides a convenient declarative abstraction, its default concurrent map implementation lacks native support for Time-To-Live (TTL) expiration policies, which can lead to stale data and memory bloat over time. Caffeine was selected as the optimal implementation because it allows explicit, fine-grained configurations for temporal eviction rules like TTL, ensuring data freshness and automatic memory eviction. Furthermore, compared to a distributed alternative like Redis, Caffeine functions directly within the Java Virtual Machine (JVM) memory space. For a single-instance deployment scale, this localized approach completely eliminates network I/O overhead over TCP/IP, avoids unnecessary infrastructure over-engineering, and maintains a highly cost-effective operational footprint.

3.2.3 Server-Sent Events via SseEmitter

To enable real-time operational communication, the platform implements the Server-Sent Events (SSE) protocol, managed natively within the backend framework via the Spring Web `SseEmitter` class [11]. This component is deployed inside the notification subsystem to establish a persistent, long-lived HTTP streaming channel from the server directly to the client browser. The primary purpose of this technology is to instantly push transactional updates, such as new order placements or order status updates to the administrative management dashboard without requiring the frontend to repeatedly trigger manual HTTP request cycles. While alternative protocols like WebSockets offer full-duplex bi-directional communication, SSE is selected as the optimal and pragmatic solution for this architecture. Because the notification workflow only requires a one-way downstream flow of data from the server to the client, utilizing `SseEmitter` completely satisfies the functional requirements while utilizing standard HTTP infrastructure, minimizing network overhead, and keeping the communication layer simple, lightweight, and highly maintainable.

3.3 Database Technology

PostgreSQL is utilized as the primary relational database management system for the platform, ensuring reliable data persistence and robust transaction management [12]. The database engine is responsible for enforcing strict data integrity and consistency, which are critical for handling sensitive e-commerce records such as financial payments, user accounts, and inventory stock levels. Although alternative solutions like MySQL or MongoDB are available, PostgreSQL is selected because of its superior efficiency in managing complex data structures and executing relational queries smoothly under concurrent user traffic.

3.4 External Services and Integrations

3.4.1 VNPay

The system integrates VNPay as its core electronic payment gateway to handle online transaction workflows securely [13]. This service provides customers with a safe and reliable checkout platform to execute digital payments during the order placement process. VNPay is selected due to its dominant market presence in Vietnam, its robust support for diverse payment methods, such as local banking apps, QR codes, and credit cards, and its perfect alignment with the digital transaction habits of the platform's target domestic users.

3.4.2 Ngrok

During the development phase, Ngrok is utilized to expose the local backend to the internet via a secure tunnel [14]. This tool is essential for handling the VNPay payment gateway integration, which relies on asynchronous Instant Payment Notification (IPN) webhooks to update order statuses. Because VNPay's external servers cannot communicate directly with a local `localhost` address, Ngrok provides a public URL to route these crucial payment callbacks back to the local application for validation and testing.

3.4.3 Cloudinary

Media asset management is offloaded to Cloudinary, a cloud-based platform utilized for securely storing and delivering product images [15]. This approach effectively removes the media storage burden from the local application server. Cloudinary is selected instead of traditional cloud storage alternatives due to its seamless integration workflow and native capabilities for optimizing and serving images efficiently to end-users.

3.4.4 Email service

The platform incorporates an SMTP-based messaging channel driven by the Spring Boot Starter Mail dependency to execute core account security functions, specifically email One-Time Password (OTP) verification and account password recovery workflows [8]. This service is responsible for dynamically generating and transmitting security tokens to authenticate user identities during sensitive account state transitions. Compared to alternative communication infrastructures such as dedicated SMS gateways or third-party messaging APIs, implementing an email-driven verification model provides a highly reliable, universally accessible, and cost-effective solution that aligns perfectly with the operational scope of the architecture.

3.4.5 EasyExcel

EasyExcel is an open-source, high-performance Java library used for reading and writing Excel files efficiently [16]. In this system, EasyExcel is integrated into the Product Manager module to support bulk product import from uploaded spreadsheet files. It helps automate data extraction, validation, and mapping of product information into the PostgreSQL database, reducing the need for manual data entry. EasyExcel is selected because it provides better memory efficiency than traditional Apache POI-based processing [17]. Instead of loading the entire Excel file into JVM memory, it processes data sequentially using a streaming mechanism. This approach keeps memory usage stable and reduces the risk of out of memory when handling large product catalog files.

3.5 Deployment Technology

3.5.1 Docker and Docker Compose

The deployment and infrastructure orchestration of the platform are managed using Docker and Docker Compose [18]. This containerization technology encapsulates the frontend application, backend services, and database engine into isolated, independent software containers. The primary purpose of utilizing Docker is to eliminate the classic issue of configuration drift and environment inconsistency between local development and live production setups. By packaging all runtime dependencies, system tools, and environment variables into a single, declarative configuration file, Docker Compose provides a lightweight and unified mechanism to run the entire multi-service stack simultaneously. This ensures that the application operates reliably, securely, and uniformly across any host machine without the overhead of heavy virtual machines or the risks of manual server configuration.

3.5.2 Nginx

Within the deployment infrastructure, Nginx is utilized as the dedicated web server embedded directly inside the React frontend Docker container to host and serve static production assets [19]. Because the React application compiles entirely into static HTML, CSS, and JavaScript files during the production build stage, a specialized web server is required to distribute these files to client browsers efficiently. NGINX is selected because it provides an exceptionally lightweight, high-performance architecture capable of handling concurrent static file delivery with minimal memory consumption. Operating within the containerized network environment, it ensures rapid response times and allows client-side routing configurations to execute smoothly, making it an ideal component for the platform's multi-container orchestration.

4.1 Architecture design

4.1.1 Software architecture selection

The system follows a three-tier client-server architecture. This architecture separates the system into three main layers: the presentation layer, the application layer, and the data layer. It is selected because it clearly separates the user interface, business logic, and data storage, making the system easier to develop and maintain.

In this project, the presentation layer is implemented using a React single-page application. It contains pages and reusable components for different actors, including Guest, Customer, Product Manager, and Administrator. The frontend communicates with the backend through RESTful APIs.

The application layer is implemented using Spring Boot. It processes business functions such as authentication, product management, cart operations, order processing, payment handling, and user management. Within this layer, the backend follows a layered structure. Controllers receive requests from the frontend, service components implement business logic, repositories access the database, and entities represent the main data models of the system.

The data layer is implemented using PostgreSQL. It stores system data such as users, products, categories, attributes, carts, orders, payments, reviews, and notifications. The separation of these three layers allows each part of the system to be maintained and updated with minimal impact on the others.

4.1.2 Overall design

This section presents the overall package design of the system through two UML package diagrams, including the frontend package diagram and the backend package diagram. These diagrams describe how the main packages are organized and how dependencies are maintained between them. The design follows a top-down dependency direction, where higher-level packages depend on lower-level packages, while lower-level packages do not depend on upper layers.

Figure 4.1 shows the package structure of the frontend application. The `router` package manages navigation and role-based route protection, then directs users to the corresponding pages. The `pages` package contains full page views for different actors, such as Customer, Product Manager, and Administrator. These pages use reusable interface elements from the `components` package, shared client-side states from the `contexts` package, and API modules from the `api` package to

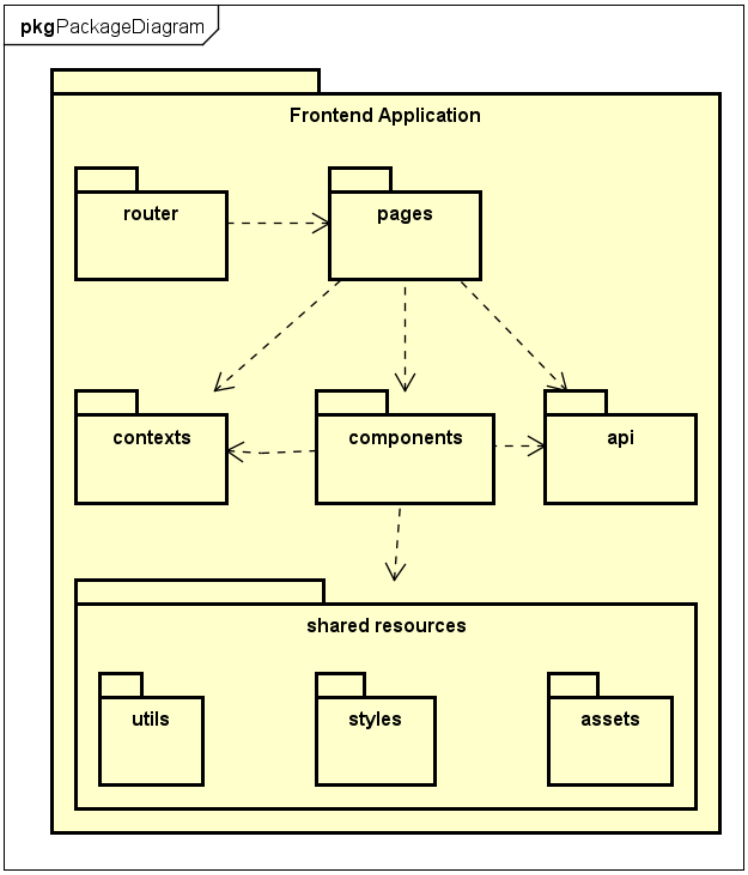


Figure 4.1: UML package diagram of the frontend application

communicate with the backend. In the diagram, the `shared resources` package is only a logical grouping and does not exist as a physical folder in the source code. It represents common frontend resources, including `utils`, `styles`, and `assets`, which provide reusable utility functions, global styles, and static images or icons. These packages support multiple frontend modules while remaining independent from specific pages or components.

Figure 4.2 presents the package structure of the backend application. When the frontend sends a RESTful request to the server, the request first passes through the security configuration in the `config` package, where authentication filters and access rules validate the token and check endpoint permissions. If the request is valid, it is forwarded to the corresponding controller.

The `controller` package receives validated requests and delegates them to the appropriate feature modules. The `feature` package contains the main business modules of the system, such as product, cart, order, payment, user, notification, and review management. Inside each feature, the `manager` package implements business logic, the `dto` package defines request and response data objects, the `repository` package handles database access, and the `entity` package rep-

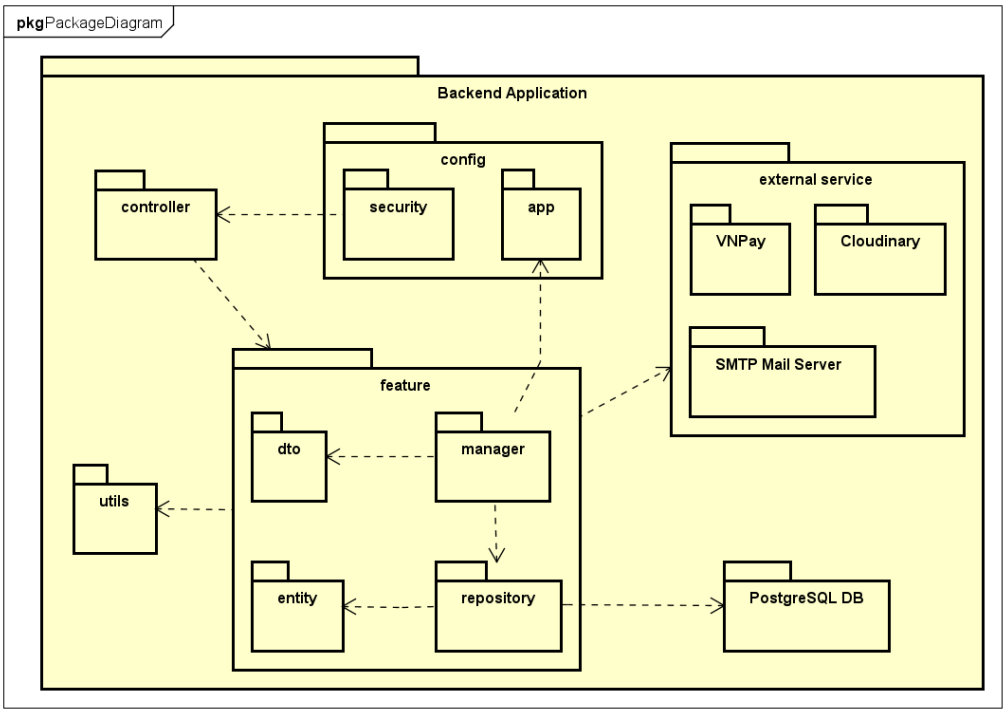


Figure 4.2: UML package diagram of the backend application

resents database models. The `exception` and `utils` packages provide common support for error handling and shared utility functions.

The backend also connects to external services and storage components. The payment feature communicates with VNPay to process online transactions, the email feature uses an SMTP mail server to send verification messages, and the media-related functions use Cloudinary for image storage. Persistent data is stored in PostgreSQL through repository components. This package organization separates the frontend, backend, database, and external services, making the system easier to maintain and extend.

4.1.3 Detailed package design

Figure 4.3 presents the detailed class design for the order placement and VNPay payment workflow. This workflow is selected because it represents one of the most important business processes of the e-commerce system, from checkout submission to payment verification and database update.

On the frontend, the routing classes direct users to the corresponding pages, such as the `Order` checkout page and the `PaymentResult` page. The `Order` page depends on reusable interface components, including `Nav`, `Header`, `Toast`, and `AddressManagement`, to build the checkout interface. It uses `OrderAPI` to send the place order request to the backend, while the `PaymentResult` page uses `PaymentAPI` to verify the payment result after VNPay redirects the user back to

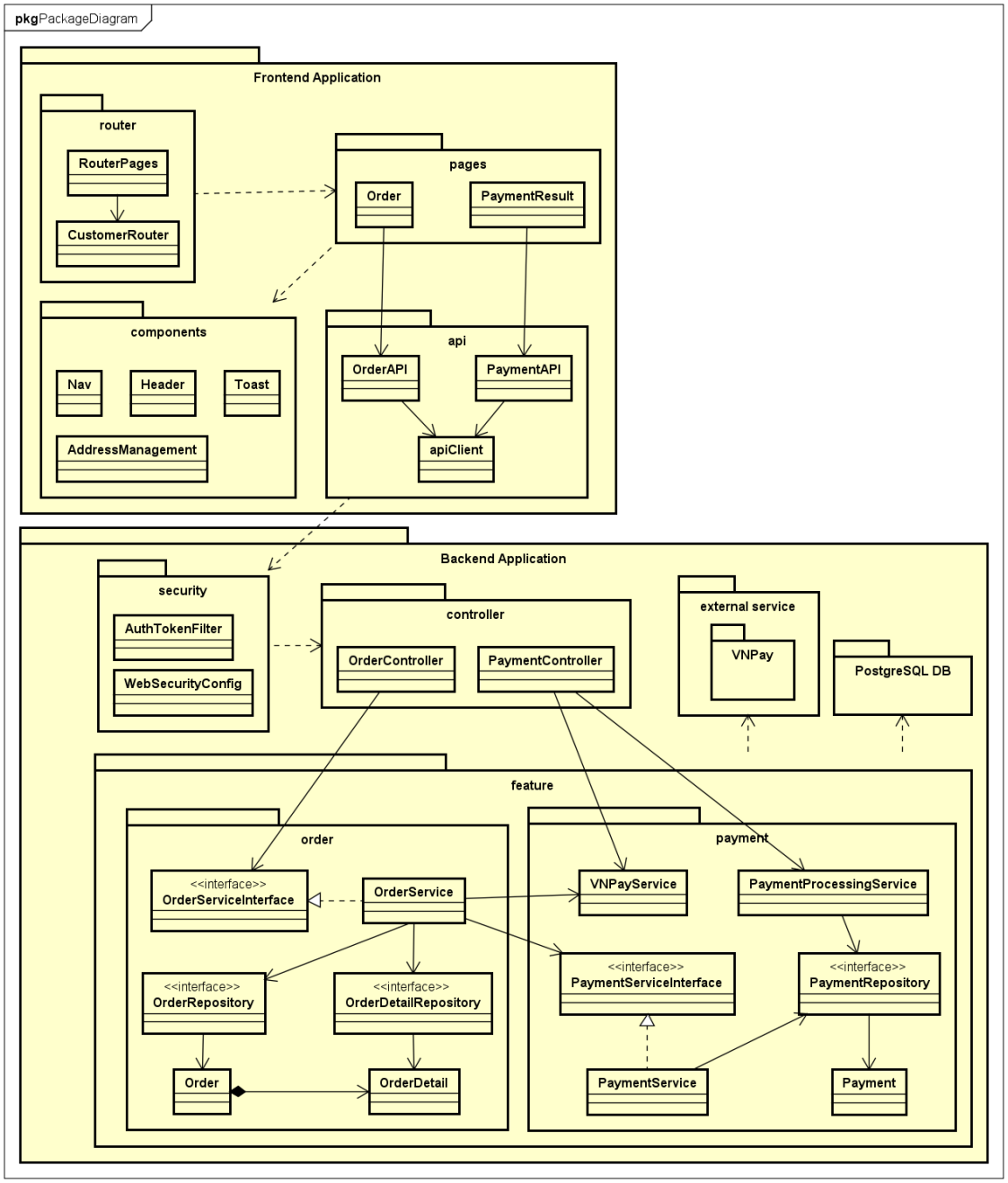


Figure 4.3: UML detailed package diagram of the order placement and VNPay payment

the system. Both API classes send HTTP requests through the shared `apiClient`.

On the backend, incoming requests first pass through the security configuration, including `AuthTokenFilter` and `WebSecurityConfig`. These classes validate authentication information and access rules before the request reaches the controllers. After that, `OrderController` delegates order placement to `OrderServiceInterface`, which is implemented by `OrderService`. The service processes order data, stores order information through repositories, creates payment information through `PaymentServiceInterface`, and uses `VNPayService` to generate the VNPay payment URL when online payment is selected.

After the customer completes payment, VNPay returns the user to the configured frontend result page and also sends the transaction result to the backend through the IPN callback. The `PaymentController` uses `VNPayService` to verify the callback data. Then, `PaymentProcessingService` updates the corresponding payment status in the database through `PaymentRepository`. In the data model, `Order` has a composition relationship with `OrderDetail`, because each order owns its item details. Other service and API interactions are represented mainly by dependency or association relationships.

4.2 Detailed design

4.2.1 User interface design

This section presents the user interface design of the e-commerce platform. The purpose of this section is to describe the target display environment, the common interface conventions, and several design mockups for important system functions. The figures presented in this section are design mockups created during the design stage.

The system is designed as a responsive web application that can be used on both desktop and mobile devices. The main target environment is the desktop browser because most management functions, such as product management, order processing, user management, and revenue monitoring, require a large working area. However, the customer-facing pages also need to support mobile screens because customers may browse products and place orders using smartphones. Table 4.1 summarizes the main screen specifications considered during interface design.

Criterion	**Specification**
Desktop resolution	`1920x1080`, `1366x768`
Mobile viewport	Responsive layout, around `390x844`
Content width	Limited main content width on large screens
Touch interaction	Sufficient size and spacing for interactive elements
Color support	True Color display, sRGB color space

Table 4.1: Target Screen Specifications

As shown in Table 4.1, the interface is mainly optimized for common desktop resolutions while still supporting smaller mobile screens. The desktop layout is important for dashboard-based screens because these pages often display tables, forms, status labels, and charts. On large screens, the main content width is constrained to prevent interface elements from stretching too widely and reducing

readability. On mobile screens, the layout adapts to smaller viewports so that important customer actions, such as browsing products, filtering products, and placing orders, remain usable. The system also assumes standard True Color display support to maintain consistent rendering of product images, icons, and interface colors.

To keep the interface consistent across different user roles, several design conventions are defined for colors, typography, buttons, and feedback messages. These conventions are applied to customer pages, product manager pages, and administrator pages. Table 4.2 lists the main interface conventions used in the system.

Element	**Design Value**	**Purpose**
Primary color	`#426B1F`	Main brand color and primary actions
Secondary color	`#6a945f`	Secondary elements and active states
Main font	`Inter`	General text and data display
Header font	`Newsreader`	Page titles and main headings
Button style	Radius `8px`, transition `0.3s`	Consistent interaction style
Feedback message	Toast notification	Quick action result display

Table 4.2: Interface Design Conventions

Table 4.2 shows the common design rules used to maintain a unified visual style. The green color scheme is used as the main identity of the platform and appears in important elements such as navigation areas and primary buttons. The `Inter` font is used for most content because it is suitable for product information, forms, and management tables, while `Newsreader` is used for headings to create a clearer hierarchy. Buttons use the same border radius and transition behavior so that users can recognize interactive elements consistently across the system. Toast notifications are used for feedback messages because they can inform users about action results without interrupting the current workflow.

After defining the target display environment and interface conventions, several wireframe mockups are prepared to illustrate the layout of important system screens. These mockups focus on the structure of the interface, including navigation areas, content placement, and major functional regions. They do not represent the final appearance of the completed product.

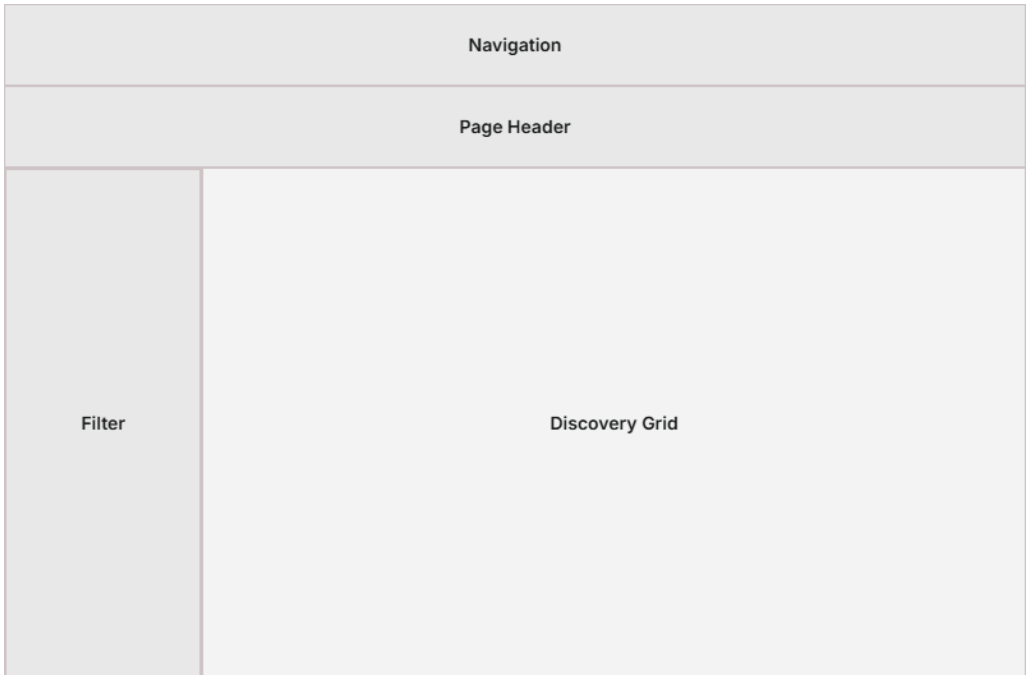


Figure 4.4: Structural wireframe of the customer product filtering interface

Figure 4.4 presents the customer product browsing interface, which is the main screen used for filtering and discovering products. The layout contains a global navigation area at the top, followed by a page header that identifies the current screen. For the product browsing page, the main content area is divided into two parts: a filter panel on the left and a discovery grid on the right. The filter panel allows customers to narrow down products based on criteria such as category, brand, price, or technical attributes. The discovery grid displays the product list after filtering, helping customers compare available options and select products for further viewing.

This layout also represents the general structure of customer-facing pages. While the top navigation area remains consistent, the lower content workspace can be adjusted depending on the function of each page. For example, the filter panel and discovery grid are used for product browsing, while other pages such as cart, order history, profile, or checkout may replace this area with tables, forms, or detailed information panels.

Figure 4.5 shows the general management layout used by both the Administrator and Product Manager pages. This layout is designed for data-heavy management tasks, such as user management, product management, order processing, review management, and revenue monitoring. The navigation area is placed as a fixed sidebar on the left, allowing management functions to remain accessible while the user works with the main content. The content workspace on the right is reserved for

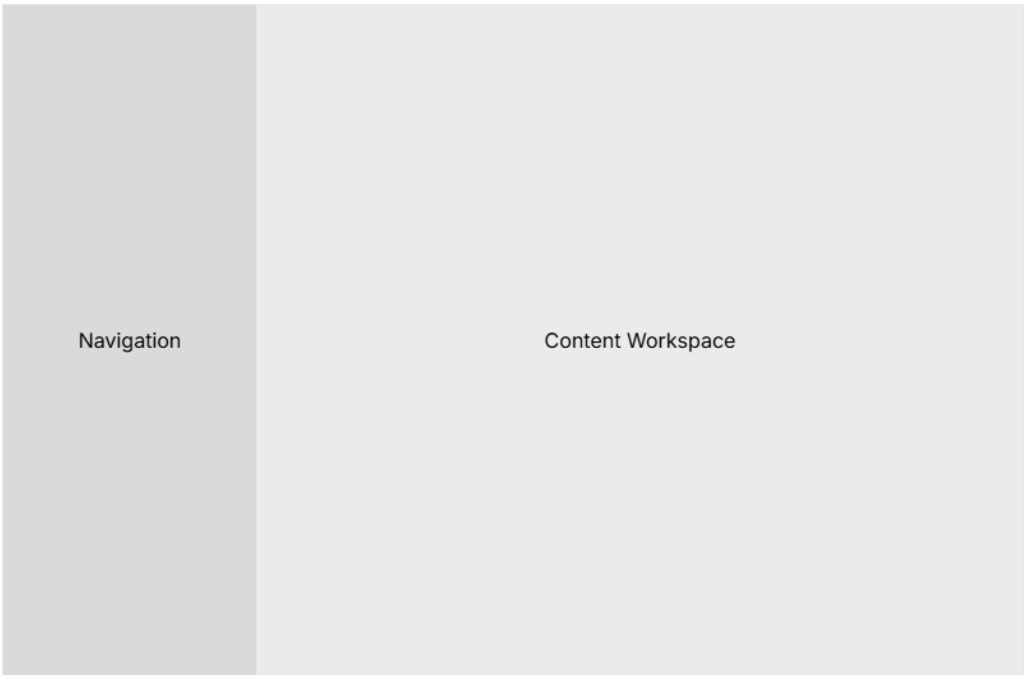


Figure 4.5: Structural wireframe of the Admin and Product Manager workspace

tables, forms, dashboards, and action controls. This structure provides enough horizontal space for management operations and keeps the interface consistent across Admin and Product Manager screens.

4.2.2 Layer design

This section presents the detailed layer design for the main classes and interaction flows related to the order and payment workflow. The selected workflow includes two important use cases of the e-commerce platform: Place Order and Pay Order. These use cases are selected because they directly affect the purchasing process, payment handling, inventory update, and order status management.

Figure 4.6 presents the detailed class design of three main service classes used in the order and payment workflow, including `OrderService`, `VNPayService`, and `PaymentProcessingService`. These classes are placed in the backend service layer and are responsible for handling business logic.

The `OrderService` class is responsible for processing order-related operations. Its main method is `placeOrder()`, which receives order information from the controller, validates the customer cart, checks product availability, calculates delivery fees, creates the order record, and coordinates with the payment module. Besides placing orders, this class also supports viewing order history, retrieving order details, cancelling orders, continuing payment, and converting order entities into data transfer objects.

The `VNPayService` class contains the logic specific to the VNPay payment

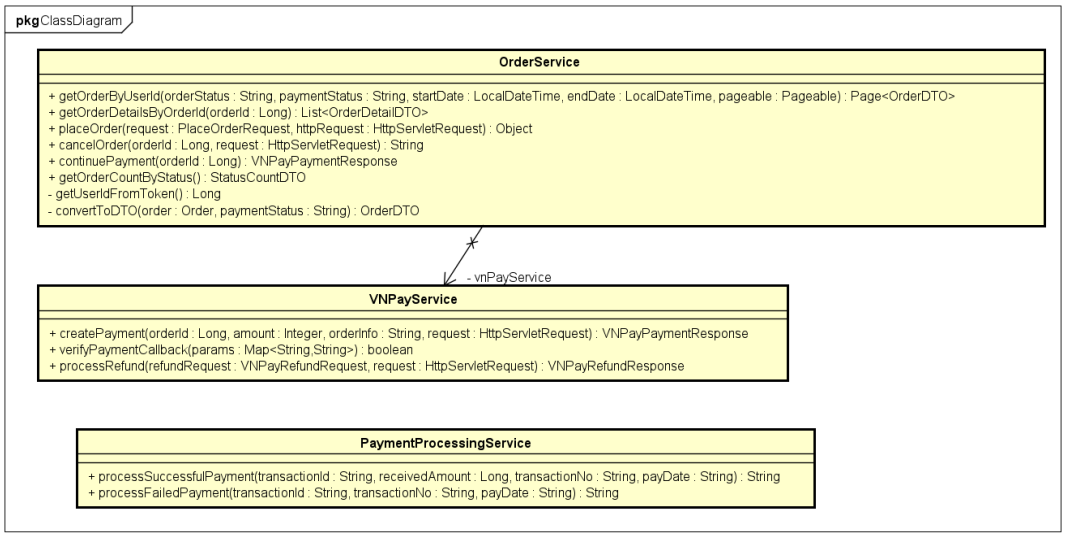


Figure 4.6: Class design for order and payment services

gateway. It is responsible for creating the VNPay payment URL, verifying callback parameters returned from VNPay, and processing refund requests when required. By separating VNPay-specific logic into a dedicated service, the system keeps the payment gateway integration independent from the general order processing logic.

The `PaymentProcessingService` class handles the final processing of VNPay transaction results. This class is responsible for updating payment status after the IPN callback has been verified. For successful payments, it checks the transaction amount and current payment status, marks the payment as paid, and clears the purchased items from the customer's cart. For failed payments, it marks the payment as failed, restores the reserved stock, and cancels the pending order if necessary. These operations are executed within a transactional boundary to maintain data consistency.

Figure 4.7 illustrates the sequence diagram for the Place Order use case. This use case starts when the customer submits the checkout form. The request is sent to `OrderController`, which forwards it to `OrderService` for business processing.

As shown in Figure 4.7, `OrderService` first identifies the current user and retrieves the customer's selected cart items. It then checks whether the cart is empty and verifies each selected product. During this process, the service checks product status, available stock quantity, and selected item quantity. If the cart and products are valid, the service decreases the product stock, calculates the delivery fee, creates the order record, and creates the corresponding payment record.

The main database-changing operations in the Place Order workflow are treated

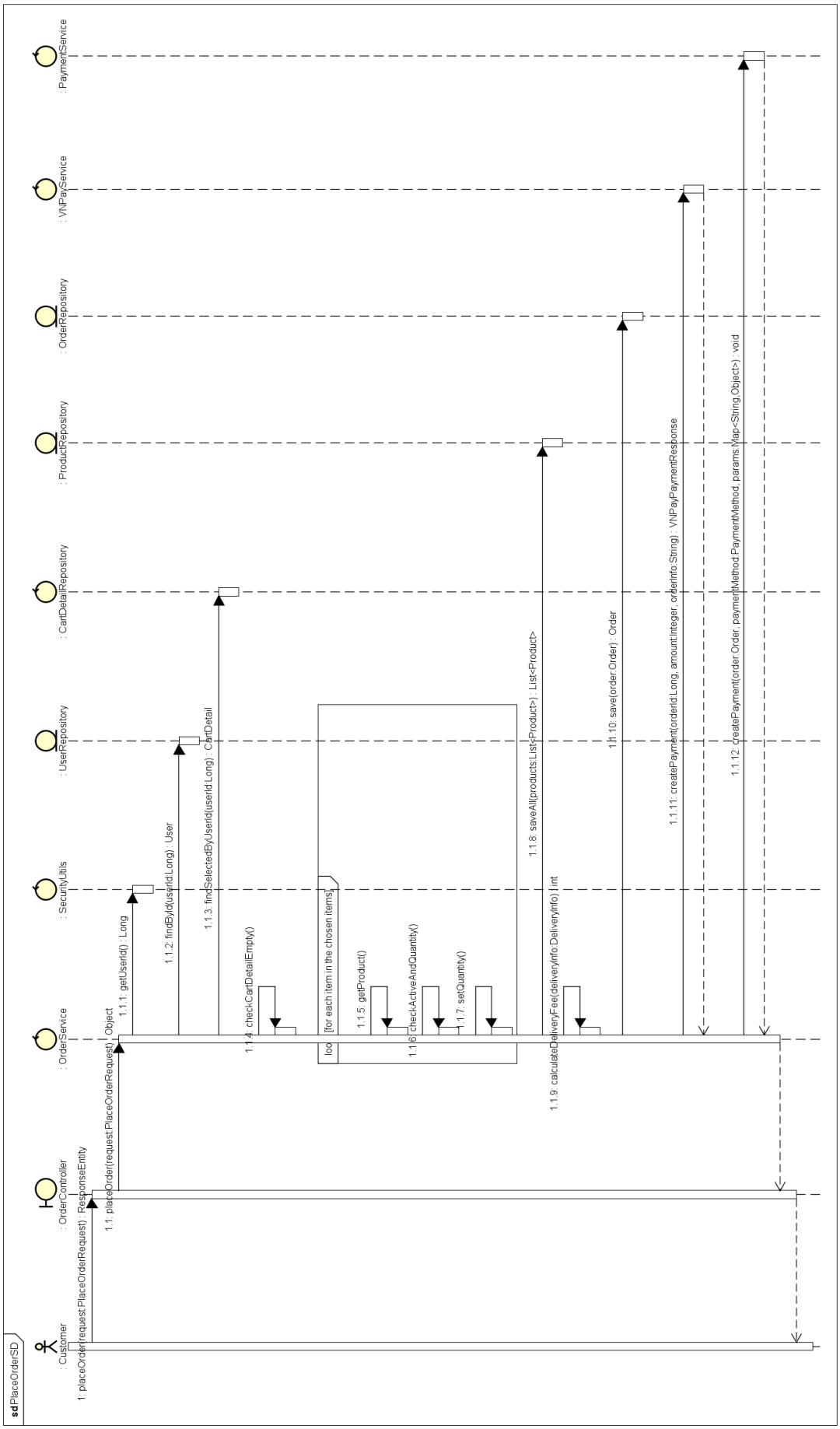


Figure 4.7: Sequence diagram for the Place Order use case

as one order-placement transaction. These operations include validating the selected cart items, decreasing stock quantity, saving the order, and creating the initial payment record. If an error occurs before the transaction is completed, these database changes should not be partially persisted. This transaction boundary prevents inconsistent states, such as an order being created without a payment record or stock being reduced without a valid order.

After the order and payment record are created successfully, the payment flow is initialized. For orders processed via VNPay, `OrderService` calls `VNPayService` to create a payment URL and returns this URL to the frontend so that the customer can be redirected to the VNPay payment page. This design separates order placement from later payment confirmation while still keeping order creation, stock reservation, and payment initialization consistent.

Figure 4.8 presents the sequence diagram for the Pay Order use case. This use case describes how the system handles the result returned from VNPay after the customer completes the payment process.

As shown in Figure 4.8, the VNPay payment result is handled through two related flows: the return URL flow and the IPN flow. The return URL flow is triggered when VNPay redirects the customer back to the system after payment. The frontend sends the returned parameters to `PaymentController`, and the controller uses `VNPayService` to verify the secure hash. This flow is used to check the legitimacy of the returned data and display the payment result to the customer. However, it does not finalize the payment status in the database.

The IPN flow is the authoritative flow for updating persistent payment data. In this flow, VNPay sends the transaction result directly to the backend server. `PaymentController` verifies the secure hash and then delegates the result to `PaymentProcessingService`. Before updating the database, the service checks whether the payment record exists, whether the received amount matches the stored amount, and whether the payment status is still pending. The payment record is updated only when these conditions are satisfied.

The payment result processing is also protected by transaction boundaries. When the payment is successful, the system updates the payment status to paid and clears the purchased items from the customer's cart in the same transaction. When the payment fails or is canceled, the system marks the payment as failed, restores the reserved stock, and cancels the pending order in the same transaction. Therefore, payment status, stock quantity, order status, and cart data remain consistent after the callback is processed.

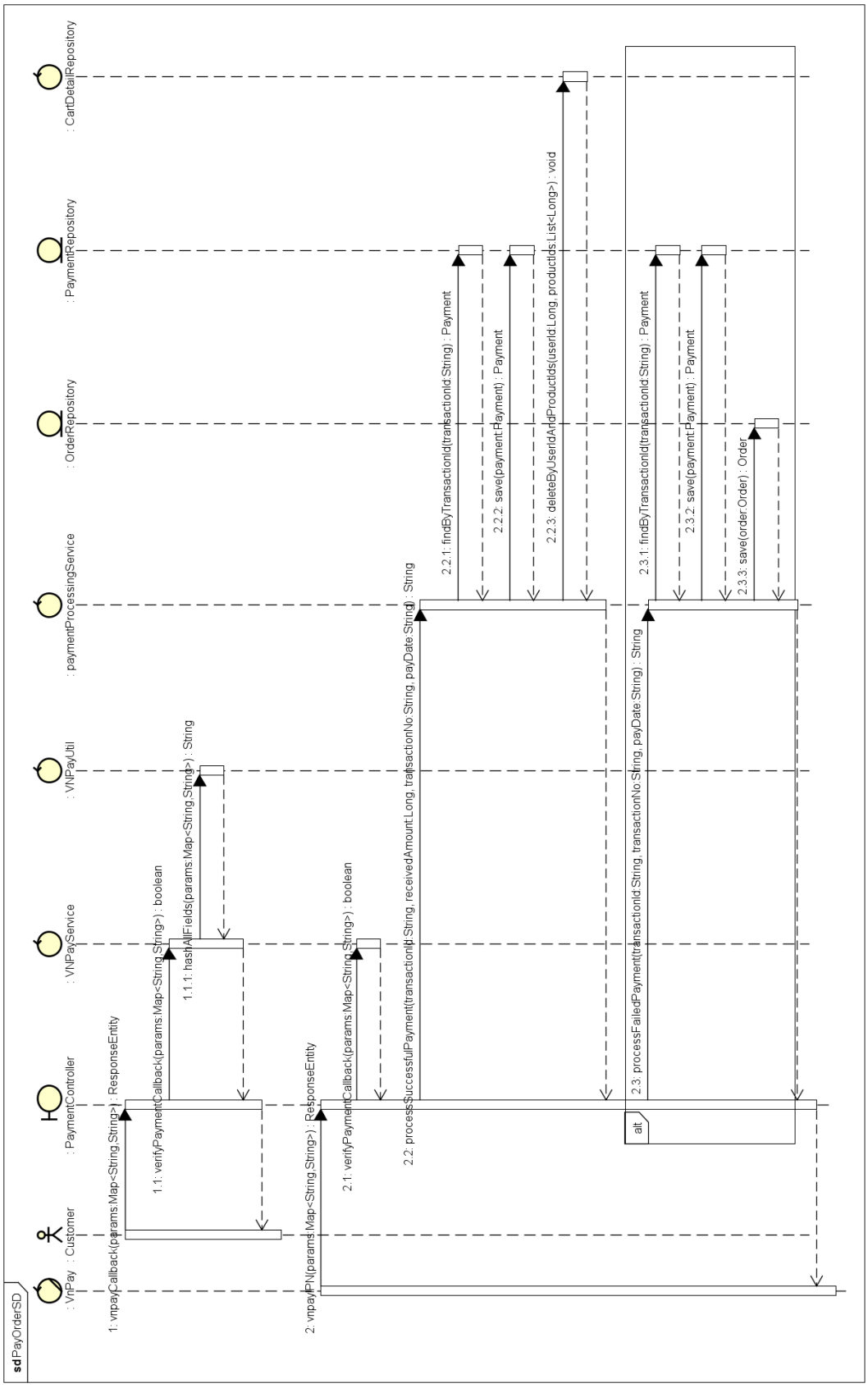


Figure 4.8: Sequence diagram for the Pay Order use case

This design defines a clear priority between the two VNPay result flows. The return URL flow is responsible for customer notification, while the IPN flow is responsible for final database updates. If the return URL arrives before the IPN, the system can display the payment result but does not finalize the database state from that flow. If the IPN arrives first, the database is updated before the customer returns to the result page. In both cases, the system only updates payments that are still pending, which prevents duplicated updates and avoids incorrect payment status changes.

4.2.3 Database design

This section presents the database design of the technology e-commerce system. The system uses PostgreSQL as the relational database management system. The database is designed based on the main business entities of the platform, including users, products, categories, carts, orders, payments, reviews, notifications, and authentication-related data. Figure 4.9 presents the overview E-R diagram of the system.

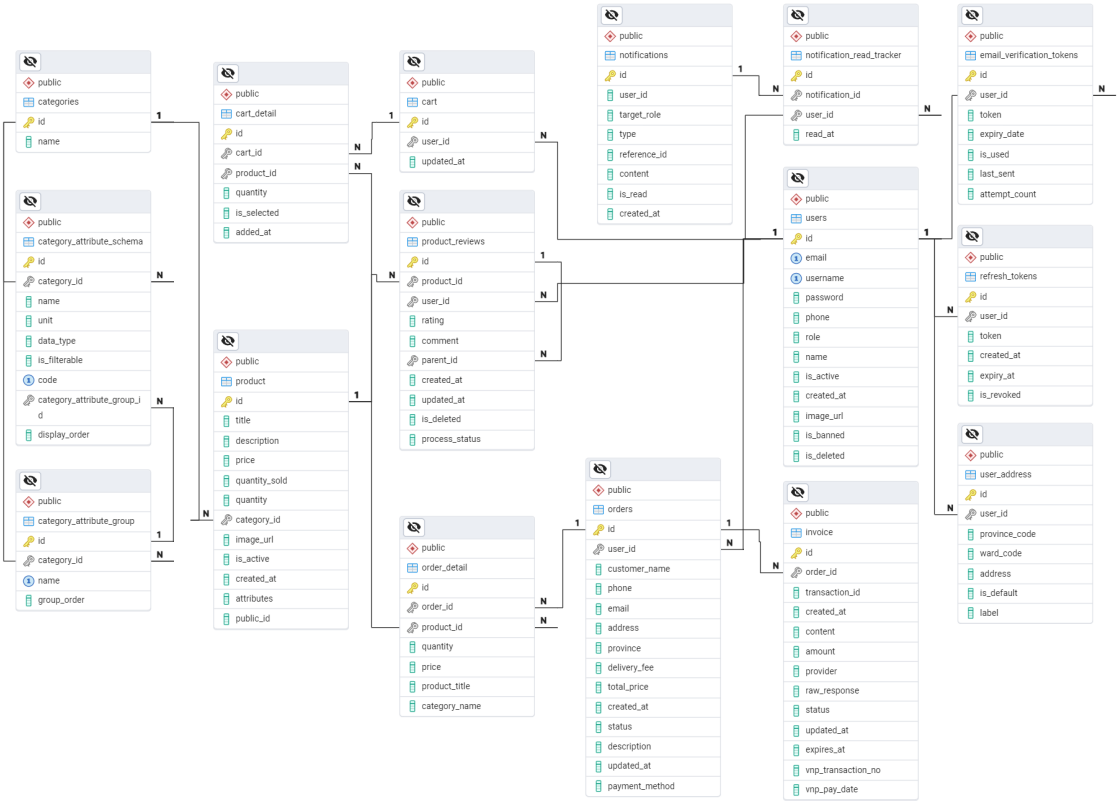


Figure 4.9: Overview E-R diagram of the technology e-commerce system

For clearer explanation, the database tables are divided into logical groups according to their responsibilities.

a, The user and authentication group

The **users** table stores account information for Customers, Product Managers, and Administrators. Table 4.3 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the user.
email	varchar(255)	Email used for login and verification.
username	varchar(255)	Username of the account.
password	varchar(255)	Encrypted password.
phone	varchar(255)	User phone number.
role	varchar(255)	Role of the user in the system.
name	varchar(255)	Full name of the user.
image_url	varchar(255)	URL of the user's profile image.
is_active	boolean	Account activation status.
is_banned	boolean	Account ban status.
is_deleted	boolean	Soft delete status.
created_at	timestamp	Time when the account was created.

Table 4.3: Design of the users table

The **user_address** table stores delivery addresses created by users. Table 4.4 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the address.
user_id [FK]	bigint	References the owner of the address.
province_code	varchar(255)	Province code of the address.
ward_code	varchar(255)	Ward code of the address.
address	varchar(255)	Detailed address information.
is_default	boolean	Indicates whether the address is the default address.
label	varchar(255)	Address label, such as home or office.

Table 4.4: Design of the user_address table

The **refresh_tokens** table stores refresh tokens used to maintain login sessions. Table 4.5 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the refresh token.
user_id [FK]	bigint	References the user who owns the token.
token	varchar(512)	Refresh token value.
created_at	timestamp	Time when the token was created.
expiry_at	timestamp	Expiration time of the token.
is_revoked	boolean	Indicates whether the token has been revoked.

Table 4.5: Design of the refresh_tokens table

The **email_verification_tokens** table stores tokens for email verification and password recovery. Table 4.6 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the verification token.
user_id [FK]	bigint	References the user being verified.
token	varchar(255)	OTP or verification token value.
expiry_date	timestamp	Expiration time of the token.
is_used	boolean	Indicates whether the token has been used.
last_sent	timestamp	Last time the token was sent.
attempt_count	integer	Number of verification attempts.

Table 4.6: Design of the email_verification_tokens table

The **notifications** table stores notification content and targeting information. Table 4.7 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the notification.
user_id [FK]	bigint	References a specific receiver when applicable.
target_role	varchar(50)	Target role for broadcast notifications.
type	varchar(50)	Type of notification.
reference_id	bigint	Identifier of the related business record.
content	text	Notification message content.
is_read	boolean	General read status for direct notifications.
created_at	timestamp	Time when the notification was created.

Table 4.7: Design of the notifications table

The **notification_read_tracker** table tracks read status for broadcast notifications. Table 4.8 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the read tracking record.
notification_id [FK]	bigint	References the related notification.
user_id [FK]	bigint	References the user who reads the notification.
read_at	timestamp	Time when the notification was read.

Table 4.8: Design of the notification_read_tracker table

b, The product and category group

The **categories** table stores product category information. Table 4.9 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the category.
name	varchar(100)	Category name.

Table 4.9: Design of the categories table

The **category_attribute_schema** table defines technical attributes for each product category. Table 4.10 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the attribute schema.
category_id [FK]	bigint	References the related category.
cag_id [FK]	bigint	References the attribute group.
name	varchar(255)	Attribute name.
unit	varchar(255)	Measurement unit of the attribute.
data_type	varchar(255)	Data type of the attribute value.
is_filterable	boolean	Indicates whether the attribute can be used for filtering.
code	varchar(255)	Attribute code used by the system.
display_order	integer	Display order of the attribute.

Table 4.10: Design of the category_attribute_schema table

The **category_attribute_group** table groups related product attributes within a category. Table 4.11 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the attribute group.
category_id [FK]	bigint	References the related category.
name	varchar(255)	Name of the attribute group.
group_order	integer	Display order of the group.

Table 4.11: Design of the category_attribute_group table

The **product** table stores the main product information displayed and managed in the system. Table 4.12 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the product.
category_id [FK]	bigint	References the product category.
title	varchar(255)	Product title.
description	varchar(5000)	Product description.
price	integer	Product price.
quantity	integer	Available stock quantity.
quantity_sold	integer	Number of sold units.
image_url	varchar(255)	Main product image URL.
is_active	boolean	Product availability status.
attributes	jsonb	Dynamic technical specifications.
public_id	varchar(255)	Cloudinary image identifier.

Table 4.12: Design of the product table

The **product_reviews** table stores customer reviews and Product Manager replies. Table 4.13 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the review.
product_id [FK]	bigint	References the reviewed product.
user_id [FK]	bigint	References the review author.
parent_id [FK]	bigint	References the parent review for replies.
rating	integer	Rating value given by the customer.
comment	text	Review content.
is_deleted	boolean	Soft delete status.
process_status	varchar(255)	Review processing status.

Table 4.13: Design of the product_reviews table

The product and category group supports product catalog management, category-based specifications, product filtering, and product reviews. The use of PostgreSQL `jsonb` in the `product` table allows different product categories to store different technical specifications without adding many fixed columns.

c, The cart group

The **cart** table stores the shopping cart owned by each customer. Table 4.14 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the cart.
user_id [FK]	bigint	References the cart owner.
updated_at	timestamp	Last update time of the cart.

Table 4.14: Design of the cart table

The **cart_detail** table stores product items inside a cart. Table 4.15 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the cart item.
cart_id [FK]	bigint	References the parent cart.
product_id [FK]	bigint	References the selected product.
quantity	integer	Quantity selected by the customer.
is_selected	boolean	Indicates whether the item is selected for checkout.
added_at	timestamp	Time when the item was added.

Table 4.15: Design of the cart_detail table

The cart group supports temporary product selection before checkout. The `is_selected` field allows customers to checkout selected cart items while keeping other products in the cart.

d, The order and payment group

The **orders** table stores confirmed order information after checkout. Table 4.16 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the order.
user_id [FK]	bigint	References the customer who places the order.
customer_name	varchar(50)	Receiver name.
phone	varchar(50)	Receiver phone number.
email	varchar(50)	Receiver email.
address	text	Delivery address.
province	varchar(20)	Delivery province.
delivery_fee	integer	Delivery fee of the order.
total_price	integer	Total order amount.
status	varchar(255)	Current order status.
payment_method	varchar(255)	Selected payment method.
created_at	timestamp	Time when the order was created.
updated_at	timestamp	Last update time of the order.

Table 4.16: Design of the orders table

The **order_detail** table stores products included in each order. Table 4.17 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the order item.
order_id [FK]	bigint	References the parent order.
product_id [FK]	bigint	References the ordered product.
quantity	integer	Ordered quantity.
price	integer	Product price at order time.
product_title	varchar(255)	Product title at order time.
category_name	varchar(255)	Category name at order time.

Table 4.17: Design of the order_detail table

The **invoice** table stores payment information related to an order. Table 4.18 presents the main fields of this table.

Field	Data Type	Description
id [PK]	bigint	Unique identifier of the payment record.
order_id [FK]	bigint	References the related order.
transaction_id	varchar(255)	VNPay transaction identifier.
content	text	Payment content or description.
amount	integer	Payment amount.
provider	varchar(20)	Payment provider name.
raw_response	jsonb	VNPay raw response.
status	varchar(255)	Current payment status.
expires_at	timestamp	Payment expiration time.
vnp_transaction_no	varchar(255)	VNPay transaction number.
vnp_pay_date	varchar(255)	VNPay payment date.

Table 4.18: Design of the invoice table

The order and payment group supports the checkout, order tracking, and payment processing workflow. Order details store product information at the time of purchase to preserve order history, while the invoice table stores payment status and provider response data for transaction verification.

Overall, the database design follows a relational structure suitable for PostgreSQL. Primary keys identify records, while foreign keys maintain relationships between users, products, carts, orders, payments, reviews, and notifications. The design also uses status fields, soft-delete fields, and flexible data types such as `jsonb` to support business workflows and technology product specifications.

4.3 Application Implementation

4.3.1 Libraries and Tools

This section presents the main tools, programming languages, frameworks, libraries, and services used to develop the technology e-commerce platform. Table 4.19 summarizes these tools.

Purpose	Tool / Library	URL
Frontend development	React 18	https://react.dev/
Frontend routing	React Router DOM v6	https://reactrouter.com/
HTTP communication	Axios	https://axios-http.com/
Frontend styling	SASS/SCSS	https://sass-lang.com/
Backend programming language	Java 22	https://www.oracle.com/java/
Backend framework	Spring Boot 3.4.0	https://spring.io/projects/spring-boot
Security and access control	Spring Security	https://spring.io/projects/spring-security
Database access	Spring Data JPA / Hibernate	https://spring.io/projects/spring-data-jpa
Database management system	PostgreSQL 16	https://www.postgresql.org/
Payment integration	VNPay Sandbox	https://sandbox.vnpayment.vn/apis/docs/thanh-toan-pay/pay.html
Image storage	Cloudinary	https://cloudinary.com/
Excel import	Apache POI / EasyExcel	https://poi.apache.org/
Development tunnel	Ngrok	https://ngrok.com/
Containerization and deployment	Docker, Docker Compose, Nginx	https://www.docker.com/
API testing	Postman	https://www.postman.com/
IDE	IntelliJ IDEA, Visual Studio Code	https://www.jetbrains.com/idea/

Table 4.19: List of development tools and libraries used in the system

As shown in Table 4.19, the system uses React for frontend development and Spring Boot with Java for backend development. PostgreSQL is used as the main database. VNPay, Cloudinary, and Ngrok support payment integration, image stor-

age, and callback testing. Docker, Docker Compose, and Nginx are used to support deployment, while Postman and IDE tools are used during development and testing.

4.3.2 Achievement

The main result of this project is a working full-stack technology e-commerce platform. The system includes a frontend application, a backend API application, a PostgreSQL database, and deployment configuration. The frontend provides user interfaces for customers, Product Managers, and Administrators. The backend handles business logic, authentication, product management, cart operations, order processing, VNPay payment integration, image storage integration, email verification, and notifications. The database stores the main business data, including users, products, carts, orders, payments, reviews, and notifications. Together, these components form a complete web-based platform that supports the main purchasing and management workflows of a technology e-commerce system.

To evaluate the implementation scale of the project, the source code and system components were measured. Table 4.20 summarizes the main implementation statistics.

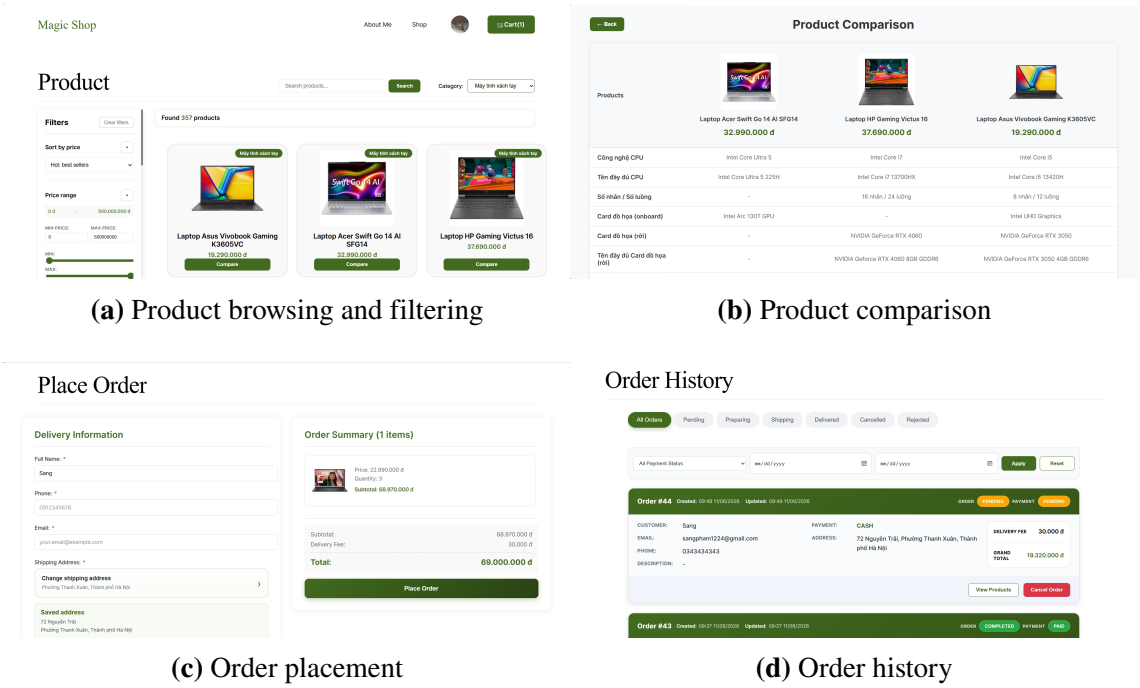
Item	**Result**
Frontend	27,808 lines of code & 157 source files & 1.13 MB source size
Backend	9,660 lines of code & 212 source files & 2.83 MB source size
Database	16 tables
API	92 REST API endpoints
User roles	4 roles: Guest, Customer, Product Manager, Administrator

Table 4.20: Implementation statistics of the system

As shown in Table 4.20, the completed system contains both client-side and server-side implementation, a relational database schema, and a REST API layer that supports multiple user roles. These results show that the project has achieved the main objective of building a functional technology e-commerce platform with product browsing, order placement, payment processing, and management features.

4.3.3 Illustration of main functions

This section illustrates the main implemented functions of the system through representative user interfaces.



(a) Product browsing and filtering

(b) Product comparison

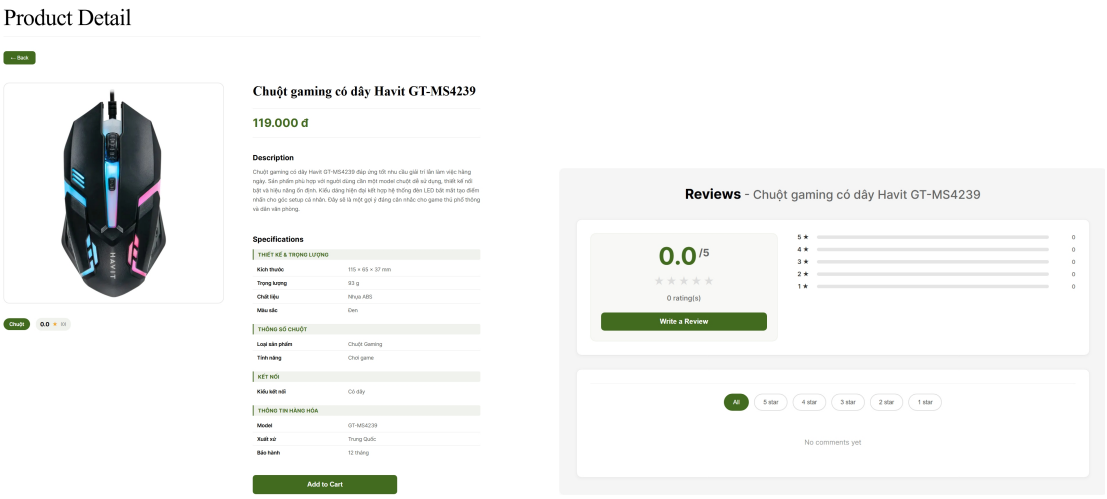
(c) Order placement

(d) Order history

Figure 4.10: Main customer interfaces of the system

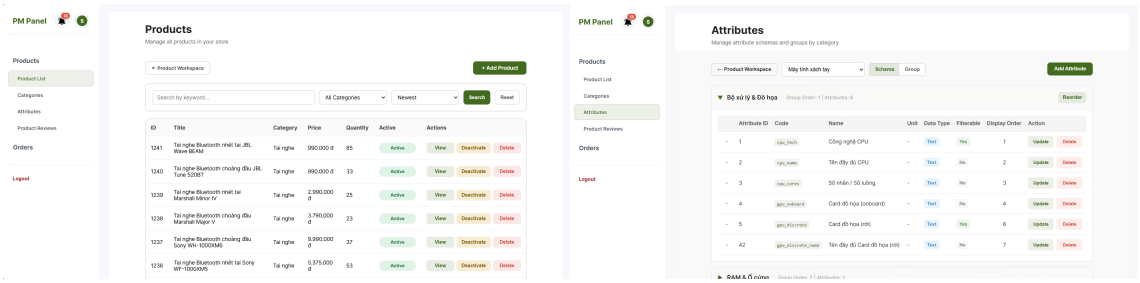
Figure 4.10 presents four main customer interfaces of the system. Figure 4.10a shows the product browsing and filtering screen, which supports product search, category selection, price filtering, and product comparison. Figure 4.10b shows the comparison screen, where customers can view selected products side by side and evaluate their technical specifications. Figure 4.10c shows the order placement screen, which collects delivery information and displays the order summary before confirmation. Figure 4.10d shows the order history screen, which allows customers to track orders, check payment status, and cancel eligible orders. These interfaces represent the main customer workflow from product discovery to order management.

Figure 4.11 presents the product detail interface of the system. Figure 4.11a shows the main product detail screen, which provides product image, price, description, technical specifications, and the add-to-cart function. Figure 4.11b shows the review section, which displays the product rating summary and customer reviews. Together, these screens help customers understand product information and evaluate feedback before making a purchase decision.

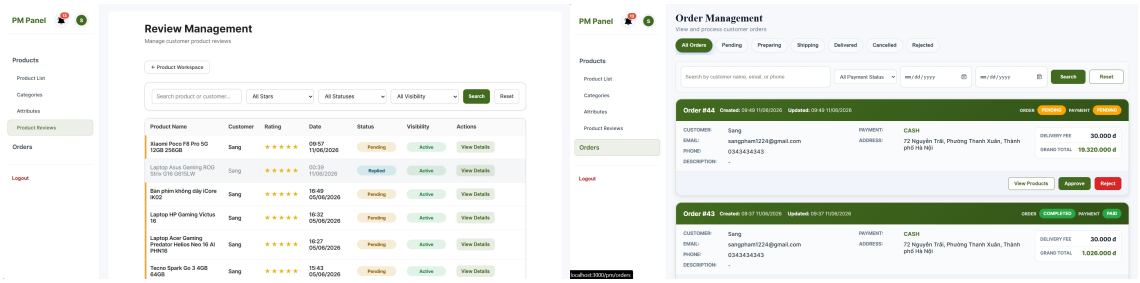


(a) Product detail information **(b)** Product review section

Figure 4.11: Product detail interface of the system



(a) Product management **(b)** Attribute management

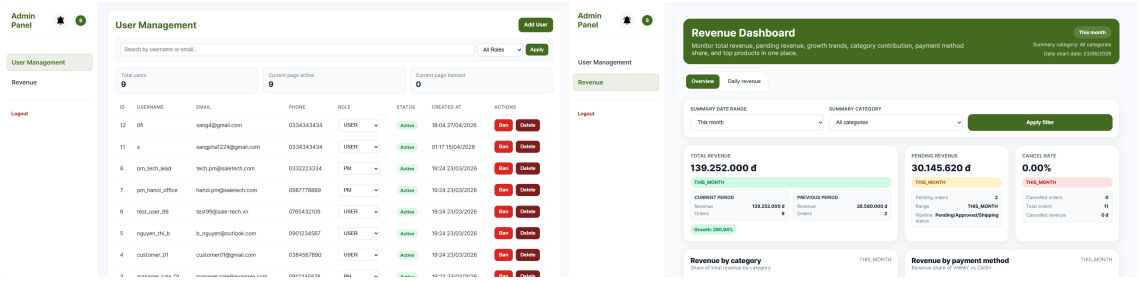


(c) Review management **(d)** Order management

Figure 4.12: Main Product Manager interfaces of the system

Figure 4.12 presents four main interfaces used by the Product Manager. Figure 4.12a shows the product management screen, where the Product Manager can view products, search by keyword, filter by category, and perform actions such as adding, viewing, deactivating, or deleting products. Figure 4.12b shows the attribute management screen, which is used to manage category-based attribute schemas and attribute groups for technology products. Figure 4.12c shows the review management screen, where customer reviews can be searched, filtered, viewed, replied to, and managed by visibility or processing status. Figure 4.12d shows the order management screen, which allows the Product Manager to view

customer orders, filter by status, inspect order details, and approve or reject pending orders. The notification bell in the Product Manager panel provides real-time alerts for important events such as new orders, payment updates, or review-related actions, helping the Product Manager respond to system activities more quickly.



(a) User management **(b)** Revenue dashboard

Figure 4.13: Main Administrator interfaces of the system

Figure 4.13 presents two main interfaces used by the Administrator. Figure 4.13a shows the user management screen, which allows the Administrator to search users, filter accounts by role, view account status, add new account, update user roles, and perform actions such as banning or deleting accounts. Figure 4.13b shows the revenue dashboard, which supports business monitoring for the Administrator. The dashboard provides total revenue and order statistics, pending revenue, cancellation rate, category-based revenue distribution, payment method revenue distribution, top-selling products, and daily revenue trends. These statistics help the Administrator evaluate sales performance, monitor unfinished or cancelled orders, and understand which product categories and payment methods contribute most to the system revenue. The notification bell in the Admin panel also helps the Administrator receive important system alerts, such as user-related events or business updates.

4.4 Testing

This section presents selected test cases for several important functions of the system. The tested functions include account registration and verification, order and payment processing, product filtering, Excel product import, access control, user management, product review, and real-time notification. These functions are selected because they affect user access, product discovery, purchasing workflow, payment consistency, data management, and system security.

The testing techniques used in this section include Equivalence Partitioning, Boundary Value Analysis, and State Transition Testing. Equivalence Partitioning is used to divide input data into valid and invalid groups, such as valid registration

information, invalid product filters, invalid Excel rows, and unauthorized requests. Boundary Value Analysis is applied to rules with clear limits, such as the maximum number of OTP attempts and the 60-second resend waiting time. State Transition Testing is used to verify whether the system updates states correctly, such as account activation, payment timeout, order cancellation, and user banning.

4.4.1 Account Registration and Verification Testing

The first tested function is account registration with email OTP verification. This function allows a guest user to create a customer account and activate it through an OTP sent by email. Table 4.21 summarizes the main test cases for this function.

Test ID	Test Case	Expected Result	Result
AR-01	Register with valid information	The system creates an inactive account and sends an OTP to the user's email.	Passed
AR-02	Register with invalid or missing input data	The system rejects the request and displays validation messages.	Passed
AR-03	Verify account with a valid OTP	The account is activated successfully.	Passed
AR-04	Enter wrong OTP 5 times	The current OTP becomes invalid and the user must request a new OTP.	Passed
AR-05	Resend OTP before 60 seconds	The system rejects the resend request and asks the user to wait.	Passed
AR-06	Resend OTP after 60 seconds	The system sends a new OTP to the user's email.	Passed
AR-07	Account is not activated within 24 hours	The system automatically removes the unverified account.	Passed

Table 4.21: Test cases for account registration and verification

As shown in Table 4.21, the registration and verification function passed all selected test cases. The results show that the system can handle valid registration, reject invalid input, control OTP verification attempts, apply resend waiting time, and remove unverified accounts after the allowed activation period.

4.4.2 Order and Payment Testing

The second tested function is the order and payment workflow. This function is selected because it directly affects the main purchasing process of the system, including order creation, stock validation, payment method handling, payment timeout, and order cancellation. Table 4.22, 4.23 summarizes the main test cases for this workflow.

Test ID	Test Case	Expected Result	Result
OP-01	Place order with valid cart, delivery information, and Cash payment	Order is created, stock is updated, cart is cleared, and notification is sent to the management side.	Passed
OP-02	Place order when product stock is insufficient	The system rejects the order and displays an insufficient stock message.	Passed
OP-03	Place order with VNPay payment method	Order and payment record are created, and the customer is redirected to the VNPay payment page.	Passed
OP-04	Complete VNPay payment successfully	The system verifies the payment result and updates the payment status as paid.	Passed
OP-05	Continue VNPay payment before the payment period expires	System allows the customer to continue the pending payment.	Passed
OP-06	Receive duplicated or repeated VNPay IPN callback	The system ignores the repeated update if the payment has already been processed.	Passed
OP-07	VNPay payment fails, is canceled, or expires while the customer remains in the VNPay page	System displays the corresponding payment result and updates the payment status as failed or canceled.	Passed

Table 4.22: Test cases for order and payment processing

Test ID	Test Case	Expected Result	Result
OP-08	Customer closes the VNPay page while the payment is still pending	After the payment period expires, the system scans and removes the expired pending payment.	Passed
OP-09	Customer cancels an order before it reaches an uncancellable state	The System cancels the order, restores the product stock quantity, and triggers a refund transaction.	Passed
OP-10	Place order with invalid form, empty cart, or inactive product	The system rejects the order and displays the corresponding error message.	Passed

Table 4.23: Test cases for order and payment processing (Continued)

As shown in Table 4.22, 4.23, the order and payment workflow passed the selected test cases. The results show that the system can create valid orders, prevent invalid order creation, handle insufficient stock, redirect customers to VNPay, process successful payments, prevent duplicated payment updates, clean up expired payments, and cancel eligible orders correctly.

4.4.3 Product Data and Import Testing

This subsection tests product-related functions, including filtering products by dynamic attributes stored in JSONB and importing product data from Excel files. These functions are selected because they directly affect product discovery and Product Manager data entry.

Test ID	Test Case	Expected Result	Result
PD-01	Filter products by category-specific JSONB attributes	The system returns products that match the selected dynamic attribute values.	Passed
PD-02	Filter products with unmatched or invalid attribute values	The system returns an empty result or ignores invalid filter values safely.	Passed
PD-03	Import products with valid basic information, category-specific attributes, and image URLs	The system validates each row, creates product records, maps dynamic attributes correctly, and stores uploaded image information.	Passed

Table 4.24: Test cases for product filtering and Excel import

Test ID	Test Case	Expected Result	Result
PD-04	Import products with missing required fields or invalid numeric values, such as price or stock	The system rejects the invalid rows and reports the corresponding validation errors without saving incorrect product data.	Passed
PD-05	Import products with attributes that do not match the selected category schema	The system reports an attribute validation error and prevents incorrect dynamic attribute data from being stored.	Passed
PD-06	Import a large Excel file containing multiple products and image URLs	The system processes the file with controlled memory usage and handles image upload asynchronously.	Passed

Table 4.25: Test cases for product filtering and Excel import (Continued)

The results of table 4.24, 4.25 show that the system can filter products using dynamic category-dependent attributes and can process Excel product import while detecting invalid input data.

4.4.4 Access Control and User Management Testing

Test ID	Test Case	Expected Result	Result
AC-01	Customer accesses Product Manager or Admin functions	The system denies access and prevents unauthorized operations.	Passed
AC-02	Product Manager accesses Admin-only user management functions	The system denies access because the Product Manager does not have Admin permission.	Passed
AC-03	Admin bans or unbans a user account	The System updates the user status, revokes the refresh token, displays an account ban notification to the user and denies further login access.	Passed

Table 4.26: Test cases for access control and user management

This subsection tests role-based access control and selected user management functions. These functions are important because the system separates permissions

between Customer, Product Manager, and Administrator roles.

The results of table 4.26 show that the system correctly enforces role-based access control and supports basic user status management by the Administrator.

4.5 Deployment

This section presents the deployment model used for the technology e-commerce system. The application is deployed in a containerized environment using Docker Compose. This approach allows the main components of the system to run together in a consistent environment, including the frontend service, backend service, database service, and supporting services for external integration. The deployment model is suitable for development and testing because it reduces manual configuration and makes the system easier to start, stop, and reproduce on another machine.

In the deployment environment, the frontend application is built into static files and served through Nginx. The backend application is packaged as a Spring Boot service and exposes RESTful APIs for the frontend. PostgreSQL is used as the database service to store application data. In addition, Ngrok is used during testing to provide a public callback URL for VNPay IPN requests. This is necessary because VNPay must be able to send payment notifications to the backend server, while the application is still running in a local or private development environment.

Component	**Deployment Form**	**Role**
Frontend service	Docker container with Nginx	Serves the built web interface to users.
Backend service	Docker container	Provides RESTful APIs and processes business logic.
Database service	PostgreSQL container	Stores users, products, carts, orders, payments, reviews, and notifications.
Ngrok service	Docker container	Provides a public URL for VNPay IPN callback during testing.
Docker Compose	Deployment configuration	Starts and connects all services in the same deployment environment.

Table 4.27: Deployment components of the system

As shown in Table 4.27, each deployment component has a clear responsibility. The frontend and backend containers form the main application, while PostgreSQL provides persistent data storage. Ngrok is not part of the core production logic, but

it is useful for testing online payment because it allows external services to reach the local backend callback endpoint. Docker Compose connects these services and helps the system run with fewer manual setup steps.

The deployment was tested in a local development environment. The required configuration values are stored in an environment file, including database credentials, JWT secret, VNPay information, Ngrok authentication token, email service configuration, and Cloudinary credentials. After configuration, the system can be started through Docker Compose and accessed through a web browser. The deployment test confirmed that the frontend can communicate with the backend, the backend can connect to PostgreSQL, product images can be uploaded to Cloudinary, OTP emails can be sent, and VNPay payment callbacks can be received through the Ngrok tunnel.

At the time of reporting, the system has not been deployed to a public production server and has not been evaluated with a large number of real users. Therefore, user traffic statistics, production response time, and load testing results are not included in this section. However, the local deployment result shows that the application can run as an integrated system and supports the main workflows required for testing, including product browsing, account verification, order placement, VNPay payment processing, notification delivery, and management functions.

5.1 Dynamic Product Attribute Design using PostgreSQL JSONB

5.1.1 Problem

One of the main difficulties in designing the product database is how to store technical specifications for different product categories. In a technology e-commerce system, products do not share the same specification structure. For example, a laptop may require CPU, RAM, storage, screen size, and GPU, while a smartphone may require chipset, battery capacity, camera, SIM support, and screen technology. Therefore, a fixed product table with many specification columns is not suitable because many columns would be empty for some categories and new columns would be required whenever a new specification appears.

A simple solution is to create a separate specification table for each product category. However, this approach makes the database difficult to maintain. Each new product category may require a new table, and changes in product specifications may require database schema changes. This design also makes product filtering and comparison more complicated because each category needs its own query logic.

Another possible solution is to use an Entity-Attribute-Value model, where each product attribute is stored as a separate row. This approach is more flexible, but it increases the number of records and requires more joins when retrieving product details. For customer-facing functions such as product detail display, product filtering, and product comparison, too many joins may make the query logic harder to understand and maintain.

Because of these limitations, the system needs a design that is flexible enough to support different product categories, but still simple enough for product display, filtering, comparison, and Product Manager operations.

5.1.2 Proposed Solution

To solve this problem, the system applies a hybrid design using PostgreSQL `jsonb`. The main idea is to separate attribute definition from attribute value storage. The structure of product attributes is managed by relational tables, while the actual technical specification values are stored in the `attributes` column of the `product` table using the `jsonb` data type.

In this design, the `category_attribute_group` table is used to group related specifications, and the `category_attribute_schema` table defines the attributes available for each product category. Each attribute definition contains

information such as attribute name, code, data type, unit, display order, and whether the attribute can be used for filtering. Based on this schema, the system can validate product attributes when a Product Manager creates, updates, or imports product data.

The `product.attributes` field stores the actual specification values of each product. Since this field uses `jsonb`, different product categories can store different sets of attributes without changing the structure of the `product` table. For example, a laptop product and a smartphone product can both be stored in the same table, but their technical specifications can follow different JSON structures according to their category schemas.

The filtering function is designed to combine normal relational filters and JSONB attribute filters. Relational filters include category, keyword, price range, active status, and sorting. Dynamic filters are applied to the `attributes` field. In the system, each dynamic attribute is defined with a specific data type in the category attribute schema. The supported data types include `number`, `boolean`, `text`, and `list`. Attributes with the `number`, `boolean`, or `text` type are stored as single-value fields in the JSONB object. For example, a laptop product may store attributes such as **"ram": 16**, **"screen_size": 15.6**, or **"cpu": "Intel Core i5"**. In contrast, attributes with the `list` type are stored as arrays because one product may have multiple values for the same specification, such as **"ports": ["USB-C", "HDMI"]**.

When customers filter products, the system receives the selected attribute codes and values from the frontend. For single-value attributes, the system checks whether the value stored under the corresponding JSONB key matches one of the selected filter values. For list-type attributes, the system checks whether the JSONB array contains at least one of the selected values. Therefore, the same filtering mechanism can support both simple specifications, such as RAM or CPU, and multi-value specifications, such as connection ports or supported features.

To improve the performance of attribute-based filtering, a GIN index is created on the `attributes` column of the `product` table. This column stores the dynamic technical specifications of each product in JSONB format.

```
CREATE INDEX idx_product_attributes
ON product
USING GIN (attributes);
```

The index is applied to the `product.attributes` field. It helps PostgreSQL optimize JSONB-based filtering when customers search for products by technical

specifications. In the service layer, the system builds filtering conditions from the selected attribute codes and values. For single-value attributes such as RAM, CPU, screen size, or battery capacity, the system checks whether the value stored under a JSONB key matches the selected filter value. A simplified SQL example is shown below:

```
SELECT *
FROM product
WHERE category_id = 1
    AND is_active = true
    AND attributes @> '{"ram": 16}'::jsonb;
```

For list-type attributes, such as connection ports or supported features, the value is stored as a JSON array. In this case, the query checks whether the array contains the selected value. A simplified example is shown below:

```
SELECT *
FROM product
WHERE category_id = 1
    AND is_active = true
    AND attributes @? '$.ports[*] ? (@ == "USB-C")';
```

These examples illustrate how product specifications stored in JSONB can still be queried at the database level. The relational schema defines which attributes are valid for each category, while the JSONB field stores the actual values. With the GIN index on `attributes`, attribute-based filtering can be optimized without changing the product table structure whenever a new technical specification is added.

5.1.3 Result

The final design makes the product database more flexible and easier to extend. New product categories and new technical attributes can be added through the category attribute schema without changing the structure of the main product table. This is important for a technology e-commerce system because product specifications often vary between categories and may change over time.

The solution also supports several important functions of the system. Product detail pages can display technical specifications dynamically based on the stored JSONB data. The filtering function can use category-specific attributes as customer-facing filter options. The comparison function can display products of the same category side by side using the schema-defined attributes.

Compared with a fixed-column design, this solution avoids a large product table with many unused columns. Compared with a pure Entity-Attribute-Value design, it reduces the need for multiple joins when displaying product details and comparing products. The use of a GIN index on the `attributes` field also provides a practical optimization direction for attribute-based filtering as the number of products increases.

However, this solution still has some limitations. JSONB is flexible, but it does not completely replace relational modeling. Some complex numeric range filters or frequently used technical attributes may still require additional expression indexes or generated columns if the dataset becomes much larger. Therefore, the current design is suitable for the scale and requirements of this thesis, but further performance tuning may be needed for a large production system.

5.2 VNPay Payment Timeout Recovery and Stock Reservation Cleanup

5.2.1 Problem

Online payment is one of the most sensitive parts of an e-commerce system because it affects order status, payment status, stock quantity, and customer experience at the same time. In the VNPay workflow, when a customer places an order using VNPay, the system creates an order, creates a pending payment record, reserves or reduces product stock, and redirects the customer to the VNPay payment page.

According to the VNPay payment flow, the payment result is returned to the system through two mechanisms: the return URL and the IPN callback. The return URL redirects the customer back to the application after the payment process so that the result can be displayed on the user interface. Meanwhile, the IPN callback is sent directly from VNPay to the backend server for payment confirmation. Both flows must be verified using secure hash validation before the system can trust the received data.

The problem occurs when the customer leaves the VNPay payment page before completing the transaction, such as closing the browser tab or stopping the payment process. In this situation, the system may not receive a completed payment result through either the return URL or the IPN callback. As a result, the payment record may remain in the pending state.

This pending state creates a data consistency problem. Since the order has already been created and the product stock has already been reserved or reduced, the product quantity may become inaccurate if the pending payment is never completed. Products may appear unavailable even though the customer did not finish

the payment. Therefore, the system needs a recovery mechanism to detect expired pending payments, restore reserved stock, and clean up the related order state.

5.2.2 Proposed Solution

The proposed solution focuses on adding a timeout recovery mechanism for VNPay payments that remain pending because the customer does not complete the payment process. When a customer places an order with VNPay, the system creates the order, reserves or reduces the related product stock, and creates a payment record with the `PENDING` status. This payment record is also assigned an expiration time to indicate the maximum period in which the customer can complete the payment.

If VNPay sends a valid IPN callback before the payment expires, the system processes the payment result normally. For a successful result, the payment status is updated to `PAID`, VNPay transaction information is stored, and the purchased items are removed from the customer's cart. For a failed or canceled result, the payment is marked as `FAILED`, the reserved stock is restored, and the pending order is canceled if necessary. Before applying any update, the system checks whether the payment is still in the `PENDING` state to prevent the same payment from being processed more than once.

The main recovery mechanism is implemented by a scheduled job. This scheduler periodically searches for VNPay payment records that are still `PENDING` but have passed their expiration time. These records represent abandoned payment sessions.

For each expired pending payment, the scheduler restores the stock quantity of all products in the related order, marks the payment as `FAILED`, and cancels the order if it is still in the pending state. These operations are executed within a transaction so that stock quantity, payment status, and order status are updated consistently. If an error occurs during cleanup, the system avoids saving only part of the recovery result.

With this design, the system does not leave unfinished VNPay payments in the pending state indefinitely. The IPN flow handles completed payment results, while the scheduler handles abandoned payment sessions when no final result is received. This combination keeps the payment workflow consistent and prevents product stock from being held by incomplete payment attempts.

5.2.3 Result

The proposed design improves the reliability of the online payment workflow. Successful, failed, canceled, and expired payment cases are handled with clear status updates.

The timeout recovery mechanism also helps maintain correct inventory data. When a customer closes the payment page and does not complete the transaction, the system can detect the expired pending payment and release the reserved stock. This prevents products from being held by incomplete payment sessions and keeps the product quantity more accurate for other customers.

Overall, the solution makes the VNPay integration more complete and practical. Instead of only redirecting customers to an external payment gateway, the system also handles abnormal payment interruption, pending payment recovery, and stock cleanup. This improves both data consistency and the stability of the purchasing workflow. A remaining limitation is that timeout recovery depends on the scheduler interval, so an expired payment may remain pending until the next scheduled scan.

5.3 Memory-Bounded Excel Product Import with Asynchronous Image Upload

5.3.1 Problem

Product management is a repetitive and time-consuming task in a technology e-commerce system. A Product Manager may need to add many products at the same time, and each product may contain basic information, category-specific technical attributes, and product images. If products are entered manually one by one, the process becomes slow and error-prone.

A simple bulk import approach is to read the entire Excel file into memory, convert all rows into product objects, upload images, and then save all products to the database. However, this approach has several limitations. First, loading the whole file into memory may consume too much memory when the file contains many rows. Second, product images may need to be uploaded to Cloudinary, and image upload is an I/O-heavy operation. If images are uploaded sequentially, the import process becomes slow. Third, imported technical attributes still need to follow the category attribute schema; otherwise, invalid or unknown attributes may be stored in the product data.

Another issue is the difference between fixed product fields and dynamic attributes. Fixed fields include product title, description, price, quantity, active status, and image URL. Dynamic fields depend on the selected category, such as

RAM, CPU, battery capacity, screen size, or connection ports. Therefore, the import mechanism must be able to separate fixed fields from dynamic attributes and validate them correctly.

5.3.2 Proposed Solution

The proposed solution uses a streaming-based Excel import mechanism with category-based attribute validation, batch saving, and asynchronous image upload. Instead of loading the entire Excel file into memory, the system reads the file row by row through an EasyExcel event listener. This allows the import process to handle product data progressively and reduces the risk of excessive memory usage when the file contains many rows.

Before reading the Excel file, the system checks whether the selected category exists and whether the category has an attribute schema. This step is necessary because imported products must follow the specification structure of the selected category. If the category does not have a configured schema, the import process is rejected because the system cannot validate the dynamic attributes of the products.

For each row in the Excel file, the system separates fixed product fields from dynamic attributes. Fixed fields include `title`, `description`, `price`, `quantity`, `is_active`, and `image_url`. These fields are mapped directly to the basic product information. Other columns are treated as dynamic attributes and are mapped into the product attribute object. This design allows the Excel template to contain both common product information and category-specific technical specifications.

The imported dynamic attributes are then validated against the category attribute schema. The schema defines the valid attribute codes and their data types. The system supports four main attribute types: `number`, `boolean`, `text`, and `list`. The `number` type is used for numeric specifications such as screen size or battery capacity. The `boolean` type is used for true/false specifications. The `text` type is used for textual specifications such as CPU name or operating system. The `list` type is used for attributes that may contain multiple values, such as connection ports or supported features. For list-type attributes imported from Excel, comma-separated values can be converted into a list before being stored.

To control memory usage and database operations, valid imported products are stored temporarily in a small batch. When the batch reaches the configured size, the system processes and saves the batch to the database. This avoids keeping all imported products in memory until the whole file has been read and also reduces the overhead of saving each product one by one.

Product image handling is also included in the batch processing step. If a product has an image URL, the system creates an asynchronous upload task to upload the image to Cloudinary. The system waits for all image upload tasks in the current batch to finish, updates the corresponding image URL and public ID, and then saves the batch of products. If an image upload fails, the error is logged and the system can continue processing the remaining products instead of stopping the entire import process.

5.3.3 Result

The implemented solution makes bulk product creation more practical for the Product Manager. Instead of creating products manually one by one, the Product Manager can prepare an Excel file containing basic product data, category-specific attributes, and image URLs, then import multiple products in one operation.

The solution also improves data correctness during import. Fixed product fields and dynamic attributes are separated clearly, and imported attributes are checked against the selected category schema before being stored. This helps prevent unknown attribute codes, and invalid data types from being saved into the product data.

Memory usage is better controlled because the system processes the Excel file row by row and saves products in batches. The import process does not need to keep the whole file or all product objects in memory at the same time, which makes it more suitable for larger product datasets than a simple full-memory import approach.

Asynchronous image upload improves the overall import workflow. Product images in the same batch can be uploaded concurrently, reducing the delay compared with uploading each image sequentially. The final product records can then store the uploaded image URL and public ID together with the imported product data.

A remaining limitation is that the Excel file still needs to follow the expected template structure. Although the system can validate required fields, attribute codes, and data types, it cannot fully guarantee the semantic correctness and consistency of imported content. For example, two values may both be valid text values but represent the same specification in different forms, such as `"RTX 4060"`, `"GeForce RTX 4060"`, or `"NVIDIA RTX 4060"`. This may lead to inconsistent product data and less accurate filtering or comparison results.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project has completed the development of a full-stack technology e-commerce website with a customer portal and a management dashboard. The system provides the essential functions of an online sales platform, including product browsing, product filtering and comparison, cart management, order processing, online payment, product management, user management, review management, notification, and revenue monitoring. Compared with a simple online store, the project focuses more on the characteristics of technology products, especially the need to manage diverse technical specifications across different product categories.

The main objective of the project has been achieved. The completed system supports multiple user roles, separates customer-facing functions from management functions, and integrates several external services such as VNPay, Cloudinary, email verification, and real-time notification. In addition, the project addressed several practical technical problems during implementation, including dynamic product attributes using JSONB, VNPay payment timeout recovery with stock cleanup, and memory-bounded Excel product import with asynchronous image upload.

Although the system is functional, it still has some limitations. The application has mainly been tested in a development environment, so production deployment, large-scale load testing, and real-user evaluation have not been fully conducted. Some business features, such as promotion management, delivery service integration, product recommendation, and semantic product search, also require further development before the system can be used in a real commercial environment.

Overall, the project provides a complete foundation for a technology e-commerce platform. Through the development process, practical experience was gained in database design, backend business logic, frontend-backend integration, role-based access control, payment processing, external service integration, and deployment preparation. These results show that the project has met its main goals and can be further improved for real-world operation.

6.2 Future work

In the future, the system should first be deployed to a production or cloud environment. This would allow the application to be evaluated under more realistic conditions, including real user access, domain configuration, HTTPS setup, database backup, and long-term operation. After deployment, load testing should be con-

ducted to measure response time, concurrent user handling, database performance, and payment workflow stability.

The deployment process can also be improved by applying CI/CD. Instead of building and deploying the system manually, an automated pipeline can be created to run tests, build Docker images, and deploy the application to the server. Although the current system already includes basic application logging in important backend components, it does not yet provide centralized log management, runtime metrics, monitoring dashboards, or alerting mechanisms. Therefore, future work can extend the existing logging into a production-ready monitoring solution to track API errors, payment failures, scheduler execution, database performance, and system resource usage.

Several business features can be added to improve the completeness of the e-commerce platform. The system can support voucher and promotion management, including discount codes, product-based discounts, order-level discounts, and time-limited sales campaigns.

The shipping module can also be improved by integrating with delivery service providers. Instead of using a simple delivery fee calculation, the system can calculate shipping fees more accurately based on customer address, product weight, delivery distance, delivery method, and shipping provider. Order tracking information can also be synchronized to help customers and managers follow the delivery process more clearly.

The product discovery experience can be further enhanced. A recommendation function can be developed to suggest suitable products based on categories, user behavior, purchase history, or similar product specifications. The search function can also be improved with more flexible or semantic search so that customers can find relevant products even when their search terms do not exactly match the product title.

Finally, the user interface and user experience should be refined through real-user feedback. Improvements can be made to mobile responsiveness, checkout flow, form validation messages, product comparison layout, notification display, and management dashboards. These improvements would make the system easier to use and more suitable for future commercial operation.

REFERENCE

[1] Shopee, *Shopee Website*. Accessed: Jun. 27, 2026. [Online]. Available: https://shopee.vn/.

[2] Shopee, *Shopee Seller Center*. Accessed: Jun. 27, 2026. [Online]. Available: https://banhang.shopee.vn/edu/article.

[3] Tiki, *Tiki Website*. Accessed: Jun. 27, 2026. [Online]. Available: https://tiki.vn/.

[4] Tiki, *Tiki Seller Center*. Accessed: Jun. 27, 2026. [Online]. Available: https://hocvien.tiki.vn/khoa-hoc/co-ban/.

[5] CellphoneS, *CellphoneS Website*. Accessed: Jun. 27, 2026. [Online]. Available: https://cellphones.com.vn/.

[6] The Gioi Di Dong, *The Gioi Di Dong Website*. Accessed: Jun. 27, 2026. [Online]. Available: https://www.thegioididong.com/.

[7] Meta Platforms, Inc., *React documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://react.dev/reference/react.

[8] Spring, *Spring boot reference documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://docs.spring.io/spring-boot/index.html.

[9] M. B. Jones, J. Bradley, and N. Sakimura, *Json web token (JWT)*, Internet Engineering Task Force. Accessed: Jun. 16, 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[10] B. Manes, *Caffeine caching library documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://github.com/ben-manes/caffeine.

[11] Mozilla Developer Network, *Server-sent events*. Accessed: Jun. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events.

[12] PostgreSQL Global Development Group, *Postgresql documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/.

[13] VNPay, *VNPay sandbox merchant portal*. Accessed: Jun. 16, 2026. [Online]. Available: https://sandbox.vnpayment.vn/apis/docs/thanh-toan-pay/pay.html.

[14] Ngrok, Inc., *Agent CLI quickstart*. Accessed: Jun. 16, 2026. [Online]. Available: https://ngrok.com/docs/getting-started.

[15] Cloudinary, *Cloudinary reference documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://cloudinary.com/documentation.

[16] Alibaba Group, *Easyexcel documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://easyexcel.opensource.alibaba.com/docs/current/.

[17] Apache Software Foundation, *Apache POI - the java API for microsoft documents*. Accessed: Jun. 16, 2026. [Online]. Available: https://poi.apache.org/.

[18] Docker, *Docker compose documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://docs.docker.com/compose/.

[19] NGINX, *NGINX documentation*. Accessed: Jun. 16, 2026. [Online]. Available: https://nginx.org/en/docs/.